



REPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique



Rapport de Projet de fin d'études

Filière : Génie Informatique

Réalisé par :

Raouia MSADDEK

Encadré par :

Encadrant académique : Nejla Essadi

Encadrant professionnel : Aymen Chouk

Sujet :
.....

figures/entreprise.png

Année Universitaire : 2025/2026

Dédicaces

Remerciements

Table des matières

	Dédicaces	i
	Remerciements	ii
	Avant-propos	viii
	Introduction générale	1
1	Contexte général	2
	Introduction	3
1.1	Organisme d'accueil	3
1.2	Cadre général du travail	3
1.3	Analyse de l'existant	3
	1.3.1 Description des plateformes similaires	3
	1.3.2 Comparaison et critique	4
1.4	Problématique	4
1.5	Solution proposée	5
1.6	Choix méthodologique : Développement Web	5
	1.6.1 Modèles classiques vs Agilité	5
	1.6.2 Comparaison des frameworks Agiles	6
	1.6.3 Présentation de Scrum (rôles et étapes)	7
	1.6.4 Langage de modélisation (UML)	8
1.7	Choix méthodologique : Intelligence Artificielle	8
	1.7.1 Le Generative AI Lifecycle	9
	1.7.2 Intégration Scrum + Generative AI Lifecycle	9
	Conclusion	9

2	Etat d’art	10
3	Analyse et spécification globale des besoins	11
	Introduction.	12
1	Identification des acteurs	12
2	Spécification des besoins	12
	2.1 Besoins fonctionnels	12
	2.2 Besoins non-fonctionnels	13
3	Backlog produit global	14
	3.1 Planification des Sprints	16
	3.2 Diagramme de GANTT.	17
4	Modélisation UML du système	17
	4.1 Diagramme des cas d’utilisation global	17
	4.2 Diagramme de classes global	19
5	Architecture de l’application	19
	5.1 Architecture Microservices et Monorepo.	19
6	Conclusion	20
4	Mise en œuvre de la Release 1 — Développement Web Full-Stack	22
	Introduction.	23
1	Sprint 1 : Authentification et Fonctionnalités Communes.	23
	1.1 Sprint Backlog	23
	1.2 Analyse du Sprint et Spécifications.	25
	1.3 Conception Technique et Diagrammes	33
	1.4 Réalisation et Interfaces Graphiques	39
	1.5 Conclusion du Sprint 1	42
2	Sprint 2 : Espace Apprenant (Student Workspace)	42
	2.1 Introduction et Sprint Backlog.	43
	2.2 Analyse du Sprint et Spécifications.	45
	2.3 Conception Technique et Diagrammes	51
	2.4 Réalisation et Interfaces Graphiques	54
	2.5 Conclusion du Sprint 2	57
	Conclusion générale	58
	Bibliographie et Webographie	59

Table des figures

3.1	Diagramme de GANTT de la réalisation du projet	17
3.2	Diagramme de cas d'utilisation global	18
3.3	Diagramme de classes global du système	19
3.4	Schéma de l'architecture Microservices avec l'API Gateway	20
4.1	Diagramme de cas d'utilisation : Authentification et Profil	26
4.2	Diagramme de cas d'utilisation « S'inscrire »	27
4.3	Diagramme de cas d'utilisation « S'authentifier »	29
4.4	Diagramme de cas d'utilisation « Vérifier son email »	30
4.5	Diagramme de cas d'utilisation « Réinitialiser son mot de passe »	31
4.6	Diagramme de cas d'utilisation « Gérer son profil »	32
4.7	Diagramme de séquence : Inscription	34
4.8	Diagramme de séquence : Connexion	35
4.9	Diagramme de séquence : Réinitialisation du mot de passe	36
4.10	Diagramme de séquence : Gestion du profil et Avatar	38
4.11	Diagramme de classes du Sprint 1	39
4.12	Interfaces d'inscription et de connexion	40
4.13	Interface de vérification email par OTP	41
4.14	Interface de réinitialisation du mot de passe (Étape 1 et 2)	41
4.15	Interface de gestion du profil	42
4.16	Diagramme cas d'utilisation global du Sprint 2	46
4.17	Diagramme de cas d'utilisation « Consulter le catalogue »	47
4.18	Diagramme de cas d'utilisation « s'inscrire au cours »	48
4.19	Diagramme de cas d'utilisation « Consulter le catalogue »	49
4.20	Diagramme de cas d'utilisation « Consulter le catalogue »	50
4.21	Diagramme de cas d'utilisation « Consulter le catalogue »	51
4.22	Diagramme de séquence : Inscription à un cours	52
4.23	Diagramme de séquence : Suivi de progression	53
4.24	Diagramme de séquence : Dashboard parent	54
4.25	Diagramme de classes : Sprint 2	54

4.26	Interface du catalogue des cours	55
4.27	Interface du dashboard étudiant	55
4.28	Interface du certificat de réussite	56
4.29	Interface du Virtual Lab (Shell UI)	56
4.30	Interface du dashboard parent	57

Liste des tableaux

1.1	Comparaison critique des plateformes existantes	4
1.2	Comparatif entre l'Approche classique et l'Approche Agile	6
1.3	Comparaison des principaux frameworks Agiles	7
1.4	Les 6 phases du Generative AI Lifecycle [3]	9
3.1	Product Backlog Global de TechKids Hub	15
3.2	Planification temporelle des Sprints	16
4.1	Sprint Backlog du Sprint 1 (Authentification)	24
4.2	Description textuelle du cas d'utilisation « S'inscrire »	28
4.3	Description textuelle du cas d'utilisation « S'authentifier »	29
4.4	Description textuelle du cas d'utilisation « Vérifier son email »	30
4.5	Description textuelle du cas d'utilisation « Réinitialiser son mot de passe »	31
4.6	Description textuelle du cas d'utilisation « Consulter le profil utilisateur »	32
4.7	Description textuelle du cas d'utilisation « Gérer son profil »	33
4.8	Sprint Backlog du Sprint 2 (Espace Apprenant et Parent)	43
4.9	Description textuelle : Consulter le catalogue des cours	47
4.10	Description textuelle : S'inscrire à un cours	48
4.11	Description textuelle : Suivre sa progression	49
4.12	Description textuelle : Accéder au Virtual Lab	50
4.13	Description textuelle : Lier et gérer les comptes enfants	51

Avant-propos

Introduction générale

Chapitre 1

Contexte général

Sommaire

Introduction	3
1.1 Organisme d'accueil	3
1.2 Cadre général du travail.	3
1.3 Analyse de l'existant	3
1.3.1 Description des plateformes similaires	3
1.3.2 Comparaison et critique.	4
1.4 Problématique	4
1.5 Solution proposée	5
1.6 Choix méthodologique : Développement Web	5
1.6.1 Modèles classiques vs Agilité.	5
1.6.2 Comparaison des frameworks Agiles	6
1.6.3 Présentation de Scrum (rôles et étapes)	7
1.6.4 Langage de modélisation (UML)	8
1.7 Choix méthodologique : Intelligence Artificielle.	8
1.7.1 Le Generative AI Lifecycle	9
1.7.2 Intégration Scrum + Generative AI Lifecycle	9
Conclusion	9

Introduction

Ce premier chapitre pose les fondements de notre Projet de Fin d'Études. Nous y définirons le cadre de notre intervention avant d'analyser sans complaisance les solutions éducatives actuelles. Cette analyse nous permettra de cerner les véritables verrous technologiques et pédagogiques que "TechKids Hub" ambitionne de lever, justifiant au passage nos choix méthodologiques.

1.1 Organisme d'accueil

1.2 Cadre général du travail

Ce projet s'inscrit dans le cadre de notre Projet de Fin d'Études (PFE) pour l'obtention du diplôme d'Ingénieur spécialité Intelligence Artificielle et Sciences des Données. L'objectif principal est de concevoir, développer et déployer de bout en bout une plateforme éducative intelligente nommée TechKids Hub, destinée aux enfants et adolescents âgés de 10 à 17 ans. Cette plateforme vise à rendre l'apprentissage de la programmation et de l'électronique plus accessible, interactif et personnalisé, en combinant un portail web complet avec un moteur d'Intelligence Artificielle Générative.

1.3 Analyse de l'existant

Il était impensable d'architecturer TechKids Hub sans comprendre précisément là où les plateformes actuelles échouent. L'idée n'est pas de réinventer la roue, mais de pallier des déficits d'architecture ou de logique pédagogique patents.

1.3.1 Description des plateformes similaires

Le marché regorge d'initiatives pour initier les jeunes à la programmation :

- **Code.org et Scratch** : Ces outils excellent pour démystifier l'algorithmique grâce à une interface visuelle par blocs. Leur limite ? Le plafond de verre. La transition vers des langages textuels y est abrupte, laissant souvent l'apprenant démuni face à un véritable éditeur de code.
- **Les MOOCs (Khan Academy, Coursera)** : Puissantes plateformes de diffusion de savoir, elles imposent néanmoins une posture fondamentalement passive. L'enfant écoute, regarde, mais interagit peu avec la matière.
- **L'utilisation brute des LLMs** : Confier un modèle comme ChatGPT à un élève pose un problème didactique sévère. Face à un blocage, l'IA a le réflexe

algorithmique de générer la solution complète. Le code est livré clé en main. L'effort intellectuel disparaît instantanément.

1.3.2 Comparaison et critique

Nous faisons face à un paradoxe technique et pédagogique. D'un côté, des parcours "taille unique" ultra-rigides. De l'autre, des intelligences artificielles surpuissantes mais destructrices pour l'apprentissage si elles ne sont pas bridées.

TABLE 1.1 – Comparaison critique des plateformes existantes

Plateforme	Points forts	Limites principales
Code.org / Scratch	Initiation accessible, ludique et visuelle.	Contenu limité aux bases, pas de transition vers le code textuel.
Khan Academy / MOOC	Contenu riche, accessible gratuitement.	Parcours statiques, aucune personnalisation, passivité de l'apprenant.
Assistants IA généralistes	Disponibilité immédiate, réponses variées.	Fournissent la solution directement (anti-pédagogique), contenu non contrôlé.

La majorité de ces solutions souffrent d'une approche uniforme qui ne s'adapte pas au rythme de chaque apprenant. Du côté des enseignants, la création de supports pédagogiques de qualité reste une tâche manuelle et fastidieuse. Ces limites justifient la conception d'une nouvelle solution.

1.4 Problématique

Bâtir un écosystème éducatif moderne ne se résume pas à faire des requêtes API vers un modèle de langage. Les véritables défis sont systémiques. La problématique de notre PFE s'articule autour de quatre axes :

- **Fiabilité et ancrage** : Comment orchestrer la génération de cours en garantissant l'absence totale d'hallucination factuelle pour l'enseignant ?
- **Cohérence pédagogique** : Quelle architecture de données permet de s'assurer que le contenu généré respecte strictement les prérequis de l'élève ?
- **Encadrement anti-triche** : Comment concevoir des agents IA capables d'agir comme des tuteurs guidant la réflexion, plutôt que comme de simples générateurs de code ?

- **Complexité logicielle** : Faire cohabiter un portail multi-utilisateurs et des flux d'IA générative exige une infrastructure scalable. Comment concevoir un tel système sans effondrement des performances ?

1.5 Solution proposée

TechKids Hub n'est pas un simple portail de cours. C'est une infrastructure dynamique, pensée dès le départ pour absorber la complexité de l'IA sans la subir. Pour les enseignants, la plateforme agit comme un accélérateur. Elle fournit des outils capables de générer et de structurer des contenus pédagogiques pertinents en un temps record. Pour les élèves, elle offre un environnement d'apprentissage où un assistant pédagogique les accompagne face à leurs erreurs de code, en orientant leur réflexion sans jamais court-circuiter leur apprentissage. Enfin, les parents disposent d'un tableau de bord lisible pour suivre cette évolution.

1.6 Choix méthodologique : Développement Web

1.6.1 Modèles classiques vs Agilité

L'ingénierie logicielle classique, symbolisée par le Cycle en V, exige des spécifications figées dès l'amorçage. Une approche sécurisante sur le papier, mais inadaptée à la réalité de notre produit. L'intégration de l'Intelligence Artificielle implique une part irréductible d'incertitude et d'expérimentation. Figer l'architecture dès le jour un aurait été une erreur stratégique majeure. L'approche Agile s'est imposée d'elle-même [1].

TABLE 1.2 – Comparatif entre l'Approche classique et l'Approche Agile

Critère	Approche classique	Approche Agile
Cycle de vie	Séquentiel, structuré en phases distinctes (planification, conception, développement, tests, déploiement).	Itératif, avec des cycles courts de développement et de livraison.
Planification	Minutieuse dès le début, rigide à respecter.	Continue, ajustée à chaque itération selon les retours.
Objectif	Fixé pour toute la durée du projet.	Flexible, révisable en réponse aux évolutions de l'environnement.
Changement	Difficile à intégrer en cours de route.	Encouragé et intégré naturellement au processus.
Équipe	Hierarchique, spécialisée par rôles précis.	Auto-organisée et pluridisciplinaire.
Communication	Formelle, via documents écrits et réunions planifiées.	Continue et directe entre tous les membres.
Livraison	Unique, en fin de projet après de longues phases.	Régulière, par incréments fonctionnels opérationnels.

1.6.2 Comparaison des frameworks Agiles

L'agilité donne la direction. Il fallait encore choisir le bon véhicule.

TABLE 1.3 – Comparaison des principaux frameworks Agiles

Critère	Scrum	Kanban	XP
Principe	Itérations courtes (Sprints) avec des rôles bien définis.	Flux de travail continu sans itérations fixes.	Excellence technique (TDD, Pair Programming).
Avantages	Livraisons régulières, feedback rapide, rigueur organisationnelle.	Flexibilité maximale, très visuel, idéal pour la maintenance.	Qualité de code élevée, très peu de bugs en production.
Inconvénients	Rituels parfois lourds si mal appliqués.	Peu structurant pour un projet développé de A à Z.	Discipline trop exigeante pour le prototypage rapide.

Bien que Kanban offre une souplesse séduisante, nous avons privilégié Scrum [1]. Ce framework apporte le rythme nécessaire pour cadencer nos développements web tout en nous laissant l'espace vital pour itérer sur nos agents IA entre chaque Sprint.

1.6.3 Présentation de Scrum (rôles et étapes)

Scrum est un framework agile conçu spécifiquement pour la gestion de projets informatiques complexes [1]. Il repose sur des cycles de développement courts, favorisant l'auto-organisation de l'équipe et la livraison régulière d'incréments fonctionnels.

Les rôles Scrum :

- **Scrum Master** : Il agit comme un véritable facilitateur et le garant de la méthodologie. Son rôle est de s'assurer de la bonne application des valeurs et principes agiles. Il coordonne les échanges, fluidifie la communication au sein du groupe et intervient pour lever les obstacles opérationnels afin de maximiser la productivité de l'équipe [1].
- **Product Owner (PO)** : Véritable pont entre les attentes du client (ou métier) et l'équipe technique, le PO porte la vision du produit. Il définit les spécifications fonctionnelles à partir des retours utilisateurs, priorise ces besoins avec l'équipe de développement, valide les fonctionnalités livrées et représente, au quotidien, la voix du client final.
- **Équipe de développement** : Ce groupe autonome et pluridisciplinaire est en charge de la conception et de l'implémentation technique. Le cadre Scrum proscrit toute hiérarchie interne au sein de cette équipe, favorisant ainsi la responsabilité

collective, indépendamment des spécialités de chacun. Une taille optimale se situe généralement entre 6 et 10 membres pour garantir une cohésion et une efficacité maximales [1].

Les étapes clés de la méthode Scrum :

— Étape 1 : Le Product Backlog

Lors de cette phase initiale, le Product Owner collabore étroitement avec les parties prenantes pour analyser et recueillir les besoins. Il répertorie l'ensemble des fonctionnalités attendues sous forme de *User Stories* au sein d'un référentiel central : le Product Backlog. Contrairement à un cahier des charges figé, ce document est vivant et s'ajuste continuellement aux évolutions du projet et du marché. L'équipe technique y puise les tâches réalisables et évalue leur ordre de priorité.

— Étape 2 : Le Sprint

L'approche Scrum se distingue par une organisation temporelle rigoureuse. L'équipe segmente les fonctionnalités à développer en cycles de travail courts et itératifs appelés Sprints (d'une durée moyenne de deux à trois semaines). Durant cette période, l'équipe s'isole des perturbations extérieures pour se concentrer exclusivement sur la réalisation et la livraison d'un incrément logiciel fonctionnel et démontrable.

— Étape 3 : Le Sprint Planning Meeting

Avant d'amorcer chaque itération, une réunion de planification est indispensable. Cet échange collaboratif s'apparente à une négociation entre le Product Owner, qui expose les exigences prioritaires issues du Product Backlog, et l'équipe technique. Cette dernière analyse les *User Stories* proposées, les découpe en tâches techniques concrètes et estime la charge de travail nécessaire. Une fois un accord trouvé, ces éléments validés constituent le *Sprint Backlog*, véritable feuille de route du Sprint à venir.

1.6.4 Langage de modélisation (UML)

La souplesse de l'Agilité ne dispense pas de rigueur conceptuelle. Pour cartographier notre architecture cloud et nos flux de données, nous nous sommes appuyés sur UML (Unified Modeling Language) [2].

1.7 Choix méthodologique : Intelligence Artificielle

Monter un backend web est un processus maîtrisé. Orchestrer un système multi-agents en production en est un autre. L'ingénierie de l'IA exige une rigueur scientifique que les méthodologies logicielles classiques ne couvrent qu'imparfaitement.

1.7.1 Le Generative AI Lifecycle

Pour sécuriser notre démarche de recherche, nous avons adopté le framework Generative AI Lens proposé par AWS [3].

TABLE 1.4 – Les 6 phases du Generative AI Lifecycle [3]

Phase	Nom	Description générale
1	Scoping	Définir le problème métier à résoudre, identifier les cas d’usage à forte valeur ajoutée.
2	Model Selection	Choisir le modèle le plus adapté selon les contraintes de performance.
3	Model Customization	Adapter le modèle au domaine métier via des techniques telles que le RAG.
4	Development and integration	Construire les composants applicatifs et les intégrer aux pipelines de données.
5	Deployment	Mettre le modèle en production dans un environnement scalable et sécurisé.
6	Continuous improvement	Surveiller les performances et itérer pour réduire les erreurs.

1.7.2 Intégration Scrum + Generative AI Lifecycle

La force de notre démarche réside dans la fusion de Scrum et du Generative AI Lifecycle. Pendant qu’une itération Scrum encadre les livrables du portail web, les phases du Generative AI Lifecycle structurent en parallèle les tâches d’expérimentation et de validation du moteur IA.

Conclusion

Ce premier chapitre a dessiné les contours de TechKids Hub. En observant les limites des systèmes éducatifs numérisés actuels, la trajectoire de notre projet s’est imposée d’elle-même. En couplant la fluidité opérationnelle de Scrum à la rigueur scientifique du Generative AI Lifecycle, nous avons structuré un environnement de développement capable de soutenir nos ambitions techniques. Avant de plonger dans les spécifications de la plateforme, le chapitre suivant s’attachera à disséquer l’État de l’art.

Chapitre 2

Etat d'art

Chapitre 3

Analyse et spécification globale des besoins

Sommaire

	Introduction	12
1	Identification des acteurs	12
2	Spécification des besoins	12
	2.1 Besoins fonctionnels	12
	2.2 Besoins non-fonctionnels	13
3	Backlog produit global	14
	3.1 Planification des Sprints	16
	3.2 Diagramme de GANTT	17
4	Modélisation UML du système	17
	4.1 Diagramme des cas d'utilisation global	17
	4.2 Diagramme de classes global	19
5	Architecture de l'application	19
	5.1 Architecture Microservices et Monorepo	19
6	Conclusion	20

Introduction

Ce chapitre définit le cadre conceptuel et architectural de la plateforme **TechKids Hub**. L'objectif est de traduire nos objectifs métier en spécifications rigoureuses tout en posant les bases de la structure globale du système.

Pour ce faire, nous présenterons d'abord l'analyse des besoins (acteurs et exigences) et la planification Agile (backlog et sprints). Ensuite, nous détaillerons la modélisation UML du système avant d'exposer les principes directeurs de l'architecture microservices retenue.

1 Identification des acteurs

Un système n'a de valeur que s'il répond précisément aux besoins de ceux qui l'utilisent. Dans l'écosystème TechKids Hub, nous avons modélisé quatre acteurs aux périmètres d'action distincts :

- **L'Étudiant (Apprenant)** : L'utilisateur final (10 à 17 ans). Son espace est pensé comme un véritable bac à sable interactif. Il y consulte ses parcours, écrit et exécute du code dans le *Virtual Lab*, et interagit avec le Tuteur IA sans jamais se sentir bloqué ni jugé.
- **L'Enseignant** : Le chef d'orchestre pédagogique. Son temps est précieux. Son rôle n'est plus de rédiger manuellement chaque ligne de cours, mais de téléverser des documents bruts, d'initier la génération par IA, puis de valider et d'affiner les modules avant leur publication.
- **Le Parent** : L'observateur bienveillant. Il nécessite de la visibilité sans intrusion. Son tableau de bord analytique traduit le temps passé et la progression technique de son enfant en métriques lisibles.
- **L'Administrateur** : Le garant de la stabilité. Il opère en coulisses pour provisionner exclusivement les comptes des enseignants, gérer les accès globaux, auditer les logs de sécurité et s'assurer que l'infrastructure route correctement le trafic.

2 Spécification des besoins

2.1 Besoins fonctionnels

Les besoins fonctionnels dictent la valeur métier de l'application. Ils sont ici segmentés de manière logique par volet d'intervention :

- **Volet Transversal (Fonctionnalités Communes)** :

- S’authentifier de manière sécurisée (Couplage JWT / vérification OTP) pour l’accès aux espaces dédiés.
- Gérer le profil personnel et les paramètres de sécurité du compte (tous les rôles).
- **Gestion des inscriptions différenciée** : Permettre l’auto-inscription sécurisée via OTP pour les Étudiants et les Parents ; restreindre l’inscription des Enseignants qui s’effectue exclusivement par invitation et création administrative préalable.
- **Volet Étudiant :**
 - Naviguer dynamiquement dans le catalogue de cours et **s’inscrire (enrollment)** aux parcours pédagogiques adaptés à son niveau.
 - **Exploiter le Virtual Lab selon le type de cours** : écrire et exécuter du code pur (par exemple scripts Python pour la Data Science) ou assembler virtuellement des composants électroniques (cartes Arduino, capteurs) avec simulation de code embarqué (C++).
 - Solliciter le Tuteur IA Socratique pour obtenir un guidage pas-à-pas lors de la résolution d’exercices.
 - Valider ses acquis via des évaluations formatives interactives.
- **Volet Enseignant :**
 - Ingérer des ressources documentaires hétérogènes (PDF, liens vidéo YouTube) dans le système.
 - Déclencher la pipeline de génération automatisée de contenu via l’architecture multi-agents.
 - Éditer le contenu généré grâce à un éditeur de texte enrichi, et gérer l’état de publication (Brouillon / Publié) des cours.
 - Superviser l’activité et la progression globale des étudiants rattachés à ses modules.
- **Volet Parent et Administrateur :**
 - **Parent** : Visualiser les statistiques d’apprentissage et le niveau d’engagement de son enfant via un dashboard analytique dédié.
 - **Administrateur** : Administrer les comptes utilisateurs (CRUD), provisionner de manière sécurisée les profils Enseignants et monitorer l’état de santé des différents microservices.

2.2 Besoins non-fonctionnels

Ces exigences définissent les contraintes d’architecture et de qualité que le système doit impérativement respecter pour être viable en production :

- **Performance et réactivité de l’IA** : Générer une réponse IA complexe prend du temps. Pour éviter de frustrer l’apprenant, le Tuteur IA doit impérativement

diffuser ses réponses en mode streaming (*Server-Sent Events*) afin de minimiser le délai d'attente perçu (*Time To First Token*).

- **Sécurité et Isolation** : Les données sensibles et les mots de passe doivent être chiffrés. Surtout, l'exécution de code utilisateur dans le *Virtual Lab* doit s'effectuer dans un environnement strictement sandboxé pour éviter toute compromission du backend.
- **Ergonomie** : L'interface, bien que complexe sous le capot, doit rester visuellement engageante, ludique et *responsive*, s'adaptant aussi bien aux tablettes qu'aux ordinateurs.

3 Backlog produit global

La gestion Agile repose sur un *Product Backlog* vivant. Il constitue le répertoire hiérarchisé de nos fonctionnalités. Pour organiser nos itérations, nous avons priorisé ces éléments selon la valeur métier (méthode MoSCoW) et leur complexité technique.

Afin de garantir une traçabilité parfaite entre les exigences fonctionnelles et les itérations de développement, chaque *User Story* est explicitement associée au Sprint dans lequel elle sera réalisée.

TABLE 3.1 – Product Backlog Global de TechKids Hub

ID	User Story	Priorité	Complexité	Sprint
US_01	En tant qu'utilisateur , je veux m'authentifier, réinitialiser mon mot de passe et gérer mon profil pour sécuriser mon accès.	Must	Moyenne	S1
US_02	En tant qu'administrateur , je veux gérer les comptes et superviser la plateforme pour garantir son bon fonctionnement.	Must	Moyenne	S3
US_03	En tant qu'étudiant , je veux consulter le catalogue, m'inscrire aux cours et accéder au Virtual Lab pour apprendre à mon rythme.	Must	Élevée	S2
US_04	En tant que parent , je veux lier et gérer les comptes de mes enfants, et consulter un dashboard analytique pour suivre leur progression.	Should	Moyenne	S2
US_05	En tant qu'enseignant , je veux créer et gérer mes cours manuellement (ajout, modification, suppression avant publication) pour avoir un contrôle total sur mon contenu.	Must	Moyenne	S3
US_06	En tant qu'enseignant , je veux importer des documents sources, consulter mes brouillons — qu'ils soient manuels ou générés par l'IA (identifiés par un badge) — et les valider avant publication.	Must	Élevée	S3
US_07	En tant qu'enseignant , je veux déclencher la génération automatique d'un cours structuré par l'IA à partir de mes documents importés, afin d'obtenir un brouillon prêt à valider.	Must	Très Élevée	S4, S5
US_08	En tant qu'étudiant , je veux interagir avec le Tuteur IA socratique pour obtenir un guidage pas-à-pas lors des exercices.	Should	Très Élevée	S6, S7

3.1 Planification des Sprints

La planification structure le rythme du projet. Conformément à notre démarche, le développement est scindé en deux Releases majeures, totalisant 7 Sprints de deux semaines chacun :

- **RELEASE 1 — L’ossature Web (Sprints 1, 2 & 3)** : Focalisée sur l’infrastructure logicielle, l’authentification sécurisée, et le déploiement des interfaces utilisateurs et des espaces de travail transactionnels.
- **RELEASE 2 — Le cœur cognitif IA (Sprints 4, 5, 6 & 7)** : Consacrée à l’ingénierie des données, l’orchestration du graphe d’agents, l’évaluation de la qualité des cours et l’implémentation de la boucle socratique.

Le tableau 3.2 résume la planification temporelle de ces itérations :

TABLE 3.2 – Planification temporelle des Sprints

Sprint	US	Objectifs et Fonctionnalités	Durée
S1	US_01	Authentification (JWT/OTP), réinitialisation du mot de passe, gestion de profil	2 sem.
S2	US_03, US_04	Espace Apprenant (Catalogue, inscription, Virtual Lab) et Espace Parent (Liaison des comptes enfants, Dashboard analytique).	2 sem.
S3	US_02, US_05, US_06	Espace Enseignant : création et gestion manuelle des cours, import de documents, consultation et validation des brouillons (manuels et IA). Espace administrateur	2 sem.
S4	US_07	IA (Génération) : Scoping, Model Selection et Customization (Data Investigation, Graphe Pédagogique, QLoRA).	2 sem.
S5	US_07	IA (Génération) : Development (Ingestion RAG, Orchestration LangGraph), Evaluation et Deployment.	2 sem.
S6	US_08	IA (Tuteur) : Scoping, Model Selection et Customization (Objectifs pédagogiques, Mémoire contextuelle, Socratic Prompting).	2 sem.
S7	US_08	IA (Tuteur) : Development (Agentic Loop), Evaluation (Filtres Anti-Triche) et Deployment (Streaming SSE).	2 sem.

3.2 Diagramme de GANTT

Afin de visualiser la répartition temporelle de nos efforts de développement et d'intégration, nous avons modélisé notre planification sous forme d'un diagramme de GANTT. Il illustre l'enchaînement logique et les dépendances entre les différents sprints tout au long du projet.

FIGURE 3.1 – Diagramme de GANTT de la réalisation du projet

4 Modélisation UML du système

La souplesse de l'Agilité ne dispense pas de rigueur conceptuelle. Le standard UML nous permet de figer visuellement les comportements et les structures de données avant d'écrire la moindre ligne de code.

4.1 Diagramme des cas d'utilisation global

Ce diagramme illustre de manière formalisée les interactions directes entre nos quatre acteurs et les processus majeurs du système, définissant ainsi les frontières et les permissions d'accès de l'application.

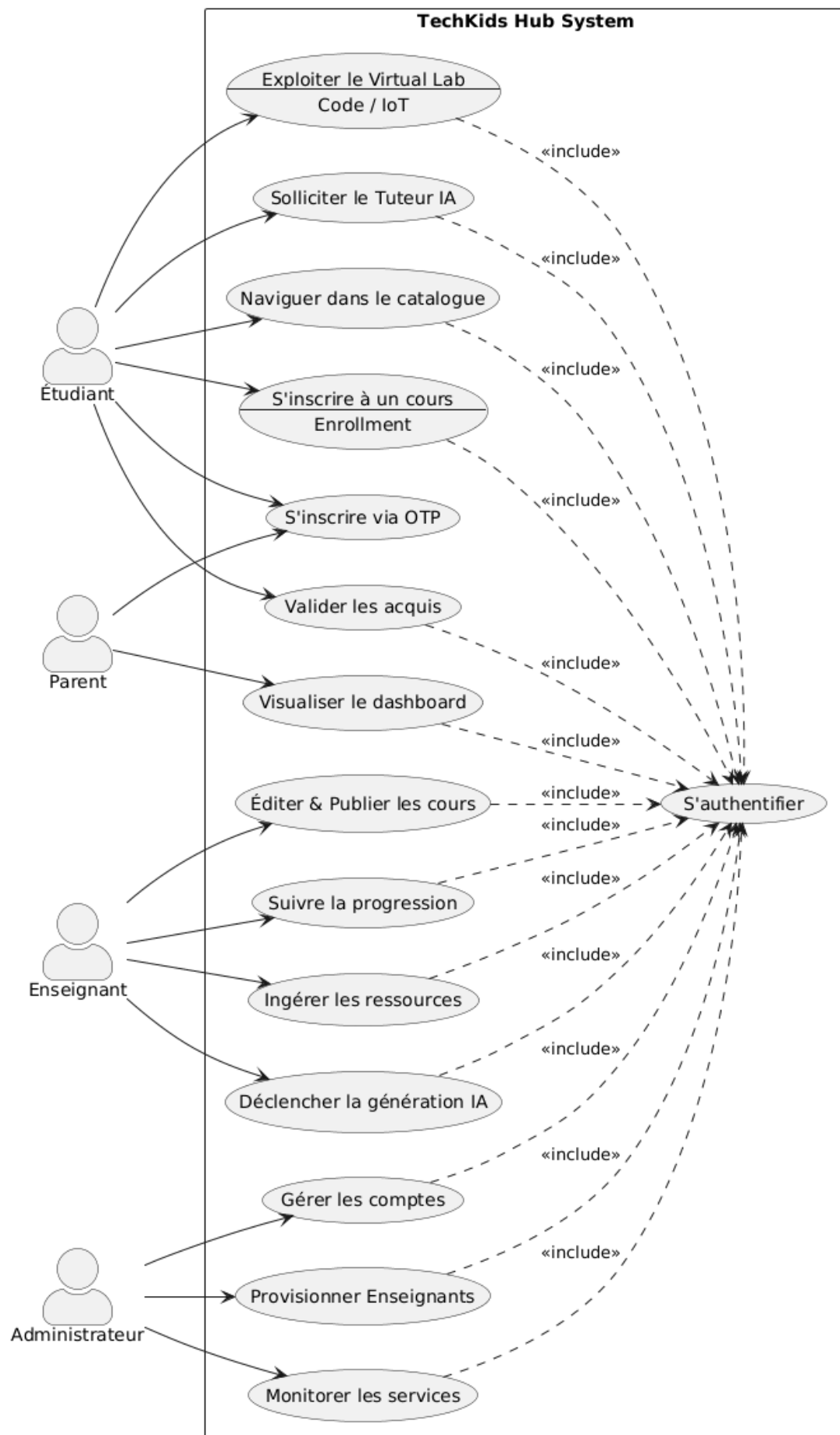


FIGURE 3.2 – Diagramme de cas d'utilisation global

4.2 Diagramme de classes global

Le diagramme de classes représente la structure statique et orientée objet de TechKids Hub. Il modélise nos entités fondamentales (Utilisateurs, Cours, Exercices, Logs d'IA), leurs attributs, et les relations de multiplicité qui les lient, constituant le squelette qui sera ultérieurement traduit en schémas de base de données.

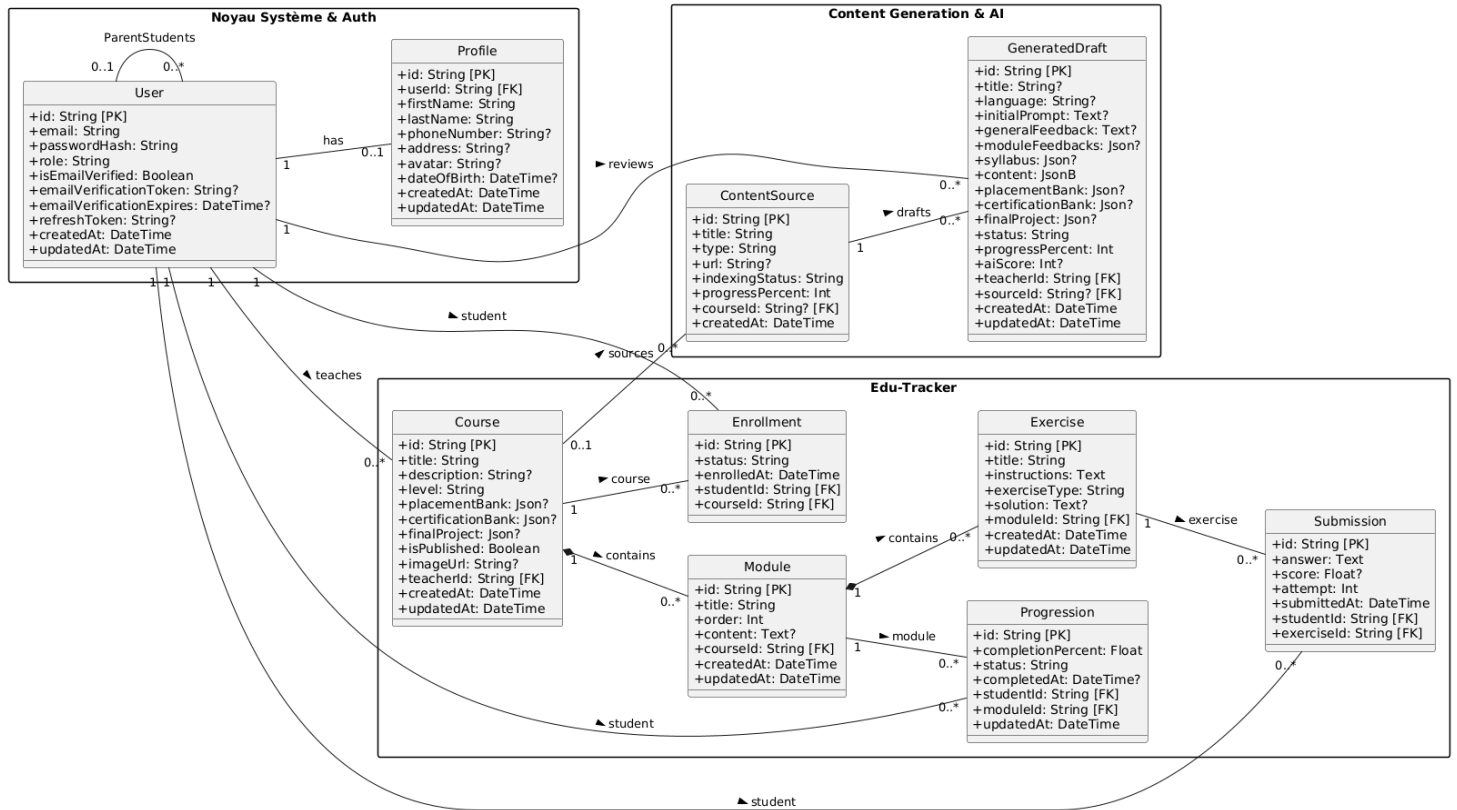


FIGURE 3.3 – Diagramme de classes global du système

5 Architecture de l'application

C'est ici que nos choix technologiques prennent tout leur sens. Pour soutenir les exigences de performance imposées par l'IA et garantir une isolation parfaite des processus, un backend monolithique aurait rapidement montré ses limites. Nous avons donc architecturé le système autour de principes modernes et scalables.

5.1 Architecture Microservices et Monorepo

La plateforme repose sur une architecture orientée **Microservices**. Afin de soutenir cette approche sans complexifier la gouvernance du projet, l'ensemble du code source (applications Front-end, services Back-end et modules d'IA) est co-localisé au sein d'un **Mono-repo** unique. Sur le plan architectural, cette stratégie offre le meilleur des deux mondes :

elle centralise la gestion des dépendances et facilite grandement le partage des structures de données et des contrats d'interface entre les équipes, tout en garantissant l'indépendance de déploiement et l'autonomie opérationnelle de chaque brique applicative.

Le trafic entrant, issu de l'Application Web, est intercepté par une **API Gateway**. Son rôle est critique : elle fait office de point d'entrée unique pour orchestrer, sécuriser et router de manière intelligente les requêtes vers quatre services autonomes et spécialisés :

- **Auth Service** : Il centralise la sécurité, la délivrance des jetons d'accès et la validation de l'identité des utilisateurs.
- **Edu-tracker Service** : Il représente le cœur métier transactionnel de la plateforme, pilotant le catalogue de cours, la gestion des ressources pédagogiques et le suivi de la progression des élèves.
- **Virtual-lab Service** : Ce service constitue le moteur d'exécution pratique. Sa conception est hybride : il gère d'une part l'exécution isolée et sécurisée de scripts de code utilisateur, et d'autre part, la simulation d'état des circuits électroniques virtuels pour les modules IoT.
- **AI Brain** : Moteur cognitif de la plateforme, ce service asynchrone est spécifiquement optimisé pour héberger l'orchestration des agents intelligents et exécuter efficacement les requêtes vers les modèles de fondation.

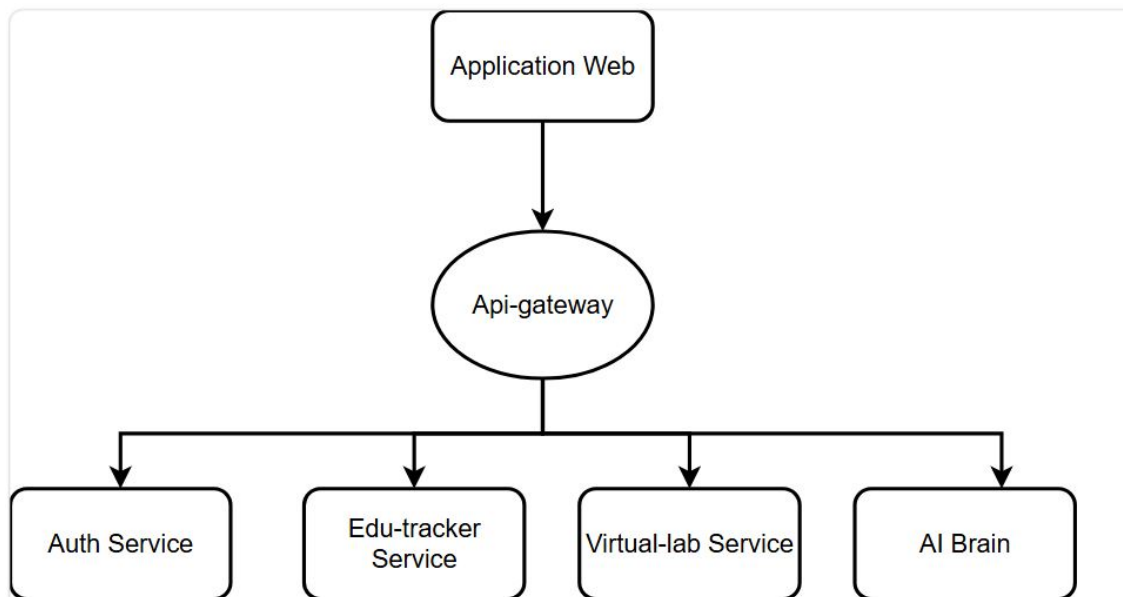


FIGURE 3.4 – Schéma de l'architecture Microservices avec l'API Gateway

6 Conclusion

Ce chapitre a permis de figer le cadre fonctionnel et architectural de la plateforme **TechKids Hub**. Grâce à l'analyse des besoins, à la planification Agile et à la modélisation

UML, le périmètre du projet est désormais clairement verrouillé.

De plus, l'adoption d'une architecture en microservices associée à une approche monorepo pose des fondations solides pour garantir la scalabilité et l'isolation du système. Ces bases étant établies, le chapitre suivant sera consacré à la réalisation pratique et à la présentation des choix technologiques retenus pour le développement.

Chapitre 4

Mise en œuvre de la Release 1 — Développement Web Full-Stack

Sommaire

	Introduction	23
1	Sprint 1 : Authentification et Fonctionnalités Communes.	23
	1.1 Sprint Backlog	23
	1.2 Analyse du Sprint et Spécifications	25
	1.3 Conception Technique et Diagrammes	33
	1.4 Réalisation et Interfaces Graphiques.	39
	1.5 Conclusion du Sprint 1	42
2	Sprint 2 : Espace Apprenant (Student Workspace)	42
	2.1 Introduction et Sprint Backlog	43
	2.2 Analyse du Sprint et Spécifications	45
	2.3 Conception Technique et Diagrammes	51
	2.4 Réalisation et Interfaces Graphiques.	54
	2.5 Conclusion du Sprint 2	57

Introduction

Suite à la spécification globale des besoins, ce chapitre est consacré à la mise en œuvre pratique de la **Release 1** de la plateforme **TechKids Hub**. Cette première grande itération se concentre sur l’ossature technique transactionnelle et le développement Web Full-Stack de l’application.

L’enjeu majeur de cette release est d’asseoir les fondations de notre architecture micro-services distribuée au sein d’un **Nx Monorepo**, et de réaliser les trois premiers sprints agiles. Nous détaillerons ainsi de manière itérative pour chaque sprint : le *Sprint Backlog*, l’analyse fonctionnelle des cas d’utilisation, la conception technique (diagrammes de séquence et sous-diagrammes de classes d’implémentation), ainsi que les interfaces graphiques finales matérialisant les espaces de travail opérationnels.

1 Sprint 1 : Authentification et Fonctionnalités Communes

Le Sprint 1 pose les fondations sécuritaires de TechKids Hub en déployant la gestion transversale des utilisateurs (inscription, connexion, récupération de mot de passe et gestion de profil). Ce socle est indispensable pour contrôler l’accès aux futurs espaces applicatifs selon les rôles.

Sur le plan technique, ce sprint s’appuie sur des échanges synchrones entre l’application web, l’**API Gateway** (qui gère le routage et la validation des jetons) et le **Auth Service**. La sécurité est renforcée par un système **OTP** obligatoire pour l’activation des comptes et la réinitialisation des mots de passe. Enfin, la gestion des profils et des données personnelles est centralisée et traitée de manière isolée par ce même **Auth Service**. t

1.1 Sprint Backlog

Ce sprint concrétise la User Story **US_01** du Product Backlog global. Lors de la cérémonie du Sprint Planning, cette exigence a été découpée en tâches techniques spécifiques dont la charge a été estimée en jours de développement.

TABLE 4.1 – Sprint Backlog du Sprint 1 (Authentification)

ID	User Story	ID	Tâche	Estimation
1	Inscription avec email, mot de passe et envoi d'un code OTP.	1.1	Réaliser les diagrammes de cas d'utilisation et séquences	3 Jours
		1.2	Développer le cas "Inscription"	
		1.3	Tester le cas "Inscription"	
2	Connexion sécurisée et redirection selon le rôle.	2.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		2.2	Développer le cas "Connexion"	
		2.3	Tester le cas "Connexion"	
3	Vérification du compte par saisie du code OTP reçu.	3.1	Réaliser les diagrammes de cas d'utilisation et séquences	1 Jour
		3.2	Développer le cas "Vérification OTP"	
		3.3	Tester le cas "Vérification OTP"	
4	Consultation du profil utilisateur et gestion des données personnelles.	4.1	Réaliser les diagrammes de cas d'utilisation et séquences	3 Jours
		4.2	Développer le cas "Gestion du profil"	
		4.3	Tester le cas "Gestion du profil"	

ID	User Story	ID	Tâche	Estimation
5	Gestion de l'avatar et suppression définitive du compte.	5.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		5.2	Développer le cas "Gestion d'avatar et suppression"	
		5.3	Tester le cas "Gestion d'avatar et suppression"	
6	Réinitialisation du mot de passe oublié par code OTP.	6.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		6.2	Développer le cas "Réinitialisation mot de passe"	
		6.3	Tester le cas "Réinitialisation mot de passe"	

1.2 Analyse du Sprint et Spécifications

L'analyse de ce premier sprint se concentre sur l'identification des processus fondamentaux d'accès et de sécurité. Pour modéliser ces interactions, nous nous appuyons sur un diagramme de cas d'utilisation global, permettant de délimiter clairement les périmètres d'action de chaque acteur avant d'entamer la conception technique.

Diagramme de cas d'utilisation global du Sprint 1

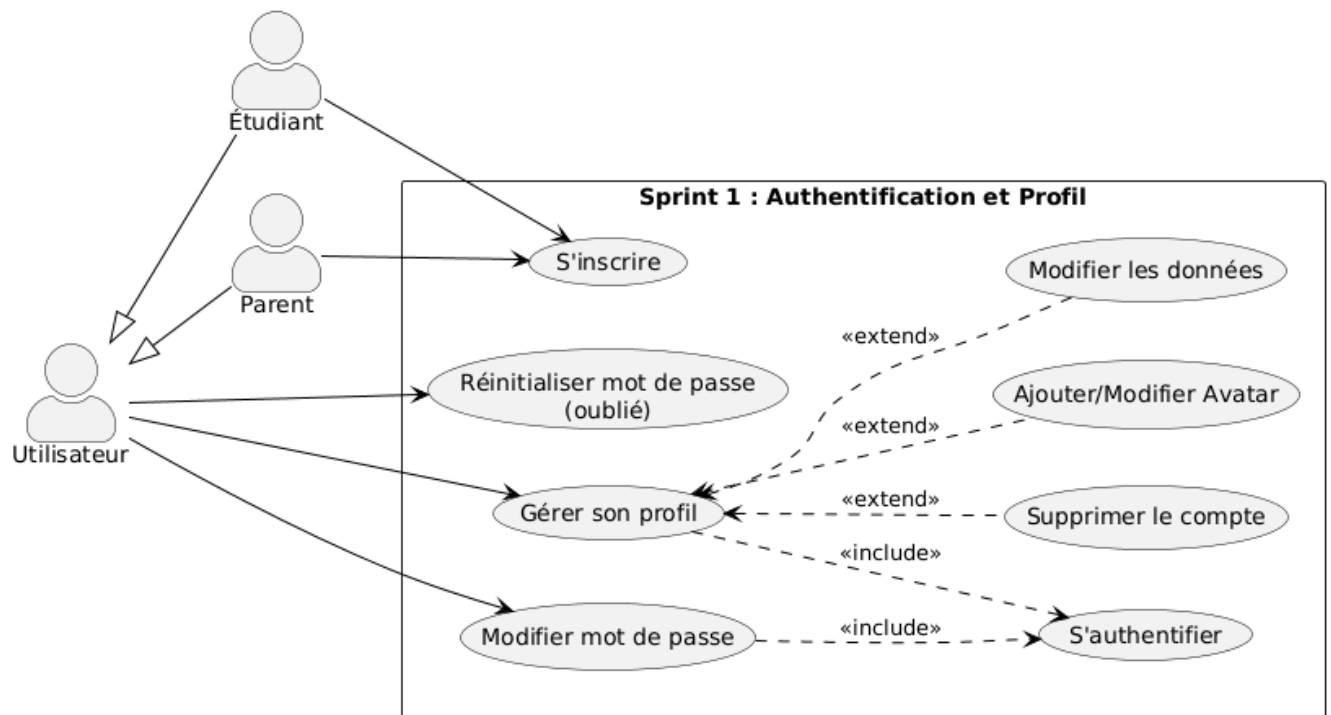


FIGURE 4.1 – Diagramme de cas d'utilisation : Authentification et Profil

Raffinement des cas d'utilisation majeurs

Voici les règles de gestion principales définies pour les cas d'utilisation de ce sprint :

- **S'inscrire** : Réservé aux étudiants et aux parents. Après la saisie des données, le mot de passe est sécurisé (hachage) et un code OTP est envoyé par email pour vérifier le compte.
- **S'authentifier** : Nécessite des identifiants valides et un compte préalablement activé via OTP. Une connexion réussie génère un jeton JWT pour sécuriser la session.
- **Gérer son profil** : Accessible uniquement aux utilisateurs connectés (*include*). Permet de modifier ses informations, d'ajouter un avatar (*extend*) ou de supprimer définitivement le compte (*extend*).

Raffinement du cas d'utilisation « S'inscrire »

La création de compte est réservée aux rôles **STUDENT** et **PARENT**. Le processus comprend la saisie des informations personnelles, leur validation côté client, la création du compte côté Auth Service et l'expédition automatique d'un code OTP à l'adresse email fournie.

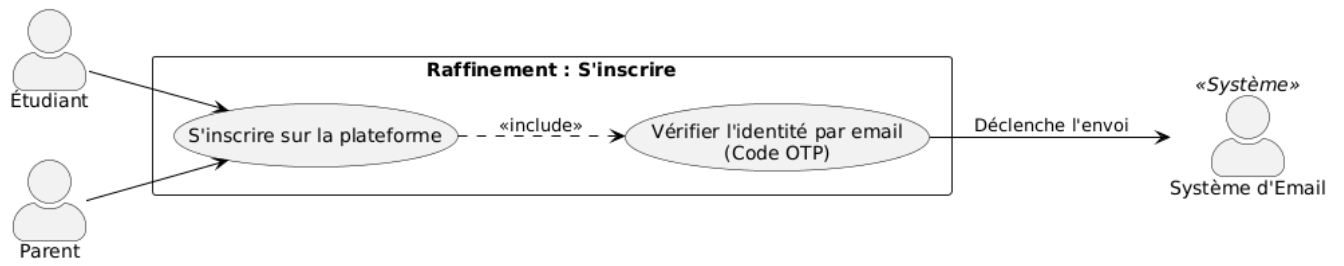


FIGURE 4.2 – Diagramme de cas d'utilisation « S'inscrire »

TABLE 4.2 – Description textuelle du cas d'utilisation « S'inscrire »

Champ	Description
Nom	S'inscrire
Acteur principal	Visiteur, futur étudiant ou parent
Acteur secondaire	Service Email
Préconditions	L'utilisateur n'est pas connecté. L'adresse email n'est pas déjà enregistrée. Le rôle choisi est STUDENT ou PARENT .
Postconditions	Le compte est créé avec <code>isEmailVerified = false</code> . Un code OTP est généré et expédié par email.
Scénario nominal	1. Le visiteur accède à la page d'inscription. 2. Il renseigne son prénom, nom, email, mot de passe et son rôle. 3. Le front transmet les données via POST <code>/api/auth/register</code>. 4. La Gateway relaie la requête au Auth Service. 5. Le service vérifie l'unicité de l'email. 6. Le mot de passe est haché avec <code>bcrypt</code> (10 rounds). 7. Un code OTP à 6 chiffres est généré. 8. Le compte est créé en base. 9. Le code OTP est envoyé à l'adresse email fournie. 10. Le service répond avec un message de confirmation. 11. L'utilisateur est redirigé vers la page de vérification.
Scénarios alternatifs	Alt 1 : Email déjà utilisé : le service retourne une erreur. Alt 2 : Email invalide ou mot de passe non conforme : le formulaire front bloque la soumission avant tout appel réseau.
Règles de gestion	RG1 : seuls les rôles STUDENT et PARENT peuvent s'auto-inscrire. RG2 : le compte reste inactif tant que l'email n'est pas vérifié. RG3 : l'OTP expire après une durée limitée.
Contraintes NF	Sécurité du mot de passe via hachage irréversible. Envoi du code dans un délai raisonnable.

Raffinement du cas d'utilisation « s'authentifier »

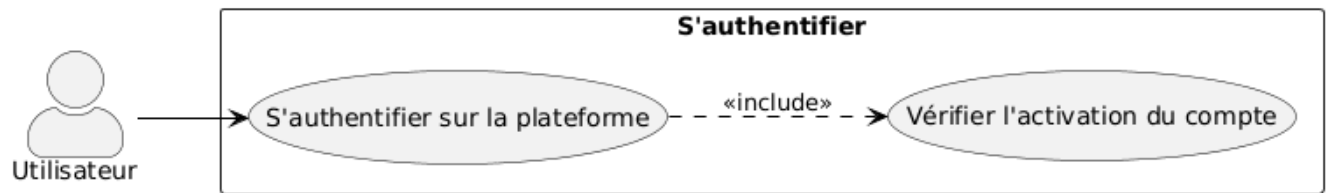


FIGURE 4.3 – Diagramme de cas d'utilisation « S'authentifier »

TABLE 4.3 – Description textuelle du cas d'utilisation « S'authentifier »

Champ	Description
Nom	S'authentifier
Acteur principal	Utilisateur inscrit et vérifié
Préconditions	Le compte existe et l'email a été vérifié avec succès.
Postconditions	L'utilisateur est authentifié. Un JWT est stocké côté client. Le profil complet est chargé.
Scénario nominal	1. L'utilisateur accède à la page de connexion. 2. Il saisit son email et son mot de passe. 3. Le front appelle POST /api/auth/login. 4. La Gateway relaie la requête au Auth Service. 5. Le service récupère l'utilisateur en base. 6. Le mot de passe est vérifié . 7. Si l'email n'est pas vérifié, la connexion est refusée. 8. Si les identifiants sont valides, un JWT est généré. 9. Le front récupère immédiatement le profil. 10. L'utilisateur est redirigé vers son espace selon son rôle.
Scénarios alternatifs	Alt 1 : Identifiants invalides : retour d'une erreur . Alt 2 : Email non vérifié : l'utilisateur est invité à compléter la vérification.
Règles de gestion	RG1 : les messages d'erreur restent volontairement génériques pour éviter toute fuite d'information.

Raffinement du cas d'utilisation « Vérifier son email »

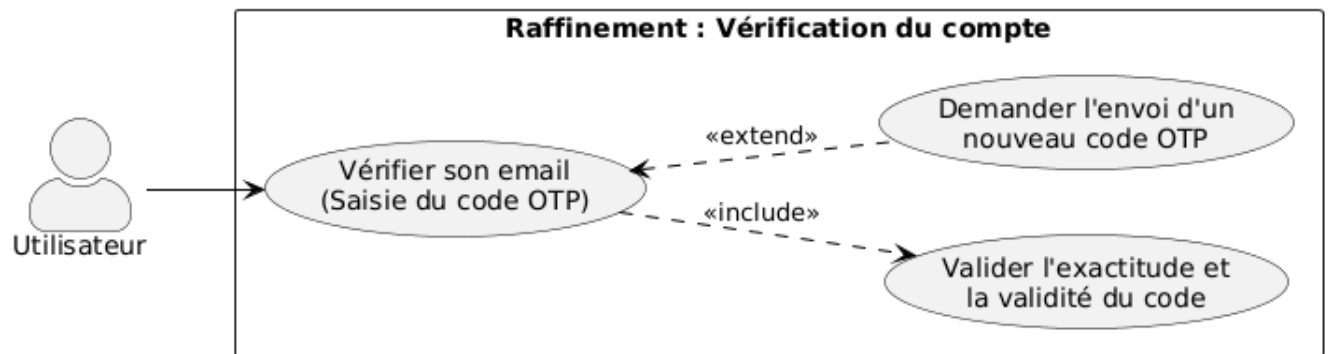


FIGURE 4.4 – Diagramme de cas d'utilisation « Vérifier son email »

TABLE 4.4 – Description textuelle du cas d'utilisation « Vérifier son email »

Champ	Description
Nom	Vérifier son email
Acteur principal	Utilisateur nouvellement inscrit
Scénario nominal	1. L'utilisateur accède à la page de vérification. 2. Il saisit le code OTP reçu. 3. Le front envoie la requête via POST <code>/api/auth/verify-email</code>. 4. Le Auth Service contrôle la validité du code. 5. Si correct, le compte est activé (<code>isEmailVerified = true</code>). 6. Les champs OTP sont effacés de la base. 7. Redirection vers la page de connexion.
Scénarios alternatifs	Alt 1 : Code invalide ou expiré : affichage d'une erreur avec possibilité de demander un nouveau code.

Raffinement du cas d'utilisation « Réinitialiser son mot de passe »

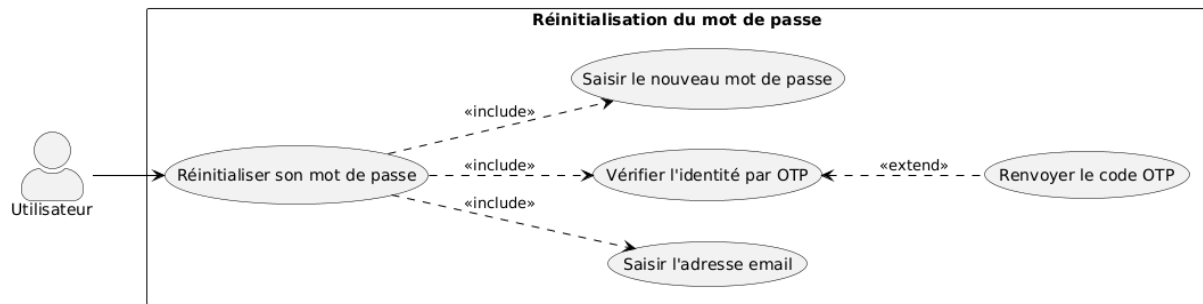


FIGURE 4.5 – Diagramme de cas d'utilisation « Réinitialiser son mot de passe »

TABLE 4.5 – Description textuelle du cas d'utilisation « Réinitialiser son mot de passe »

Champ	Description
Nom	Réinitialiser son mot de passe
Acteur principal	Utilisateur inscrit ayant oublié son mot de passe
Scénario nominal	1. L'utilisateur clique sur « Mot de passe oublié ? ». 2. Il saisit son adresse email. 3. Le service génère un code OTP et l'envoie par email. 4. L'utilisateur saisit le code reçu ainsi que son nouveau mot de passe. 5. Le service vérifie le code, hache le nouveau mot de passe, l'enregistre en base et efface l'OTP. 6. Redirection vers la connexion avec succès.
Règles de gestion	RG1 : la réponse à la demande de code est toujours identique, que l'email existe ou non, pour éviter l'énumération de comptes.

Raffinement du cas d'utilisation « Consulter le profil utilisateur »

TABLE 4.6 – Description textuelle du cas d'utilisation « Consulter le profil utilisateur »

Champ	Description
Nom	Consulter le profil utilisateur
Acteur principal	Utilisateur authentifié
Scénario nominal	1. L'utilisateur accède à la page profil. 2. Le front envoie une requête GET <code>/api/users/profile</code> avec le JWT. 3. L'API Gateway valide le token et le Auth Service interroge la base de données. 4. Les données (prénom, nom, email, rôle, avatar) sont retournées et affichées.

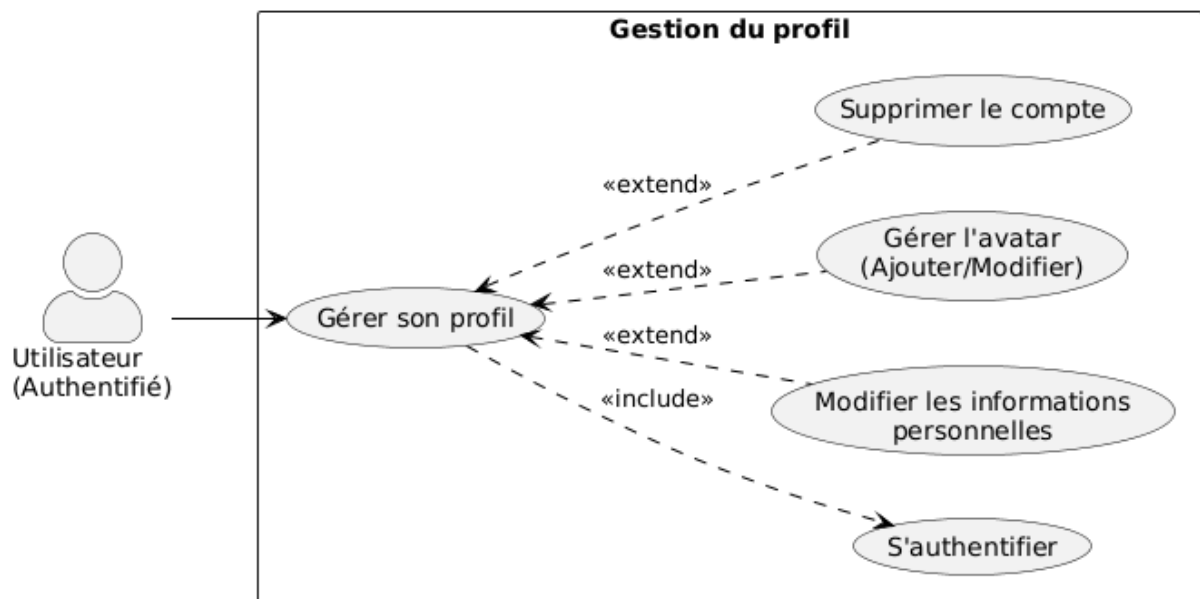


FIGURE 4.6 – Diagramme de cas d'utilisation « Gérer son profil »

Raffinement du cas d'utilisation « Gérer son profil »

TABLE 4.7 – Description textuelle du cas d'utilisation « Gérer son profil »

Champ	Description
Nom	Gérer son profil
Acteur principal	Utilisateur authentifié
Scénario nominal	1. L'utilisateur modifie ses informations personnelles (PATCH <code>/api/users/profile</code>). 2. Il peut uploader son avatar (POST <code>/api/users/avatar</code>). 3. Il peut modifier son mot de passe en fournissant le mot de passe actuel. 4. Il peut supprimer définitivement son compte. 5. Chaque modification est confirmée par un retour visuel.
Règles de gestion	RG1 : la modification du mot de passe nécessite la saisie du mot de passe actuel, vérifié par comparaison bcrypt.

1.3 Conception Technique et Diagrammes

Afin de ne présenter que les flux les plus pertinents et de ne pas alourdir le document, nous modélisons ci-dessous les diagrammes de séquence des processus les plus critiques techniquement, impliquant des algorithmes de hachage, des envois d'emails et des interactions avec des services tiers cloud.

Diagramme de séquence « S'inscrire (avec vérification OTP) »

Ce diagramme détaille l'enchaînement des échanges entre les composants du système lors de la création d'un compte. Il met en évidence le rôle de la Gateway comme point d'entrée unique, la logique de hachage exécutée par le Auth Service, et la génération ainsi que l'envoi du code OTP via le service email.

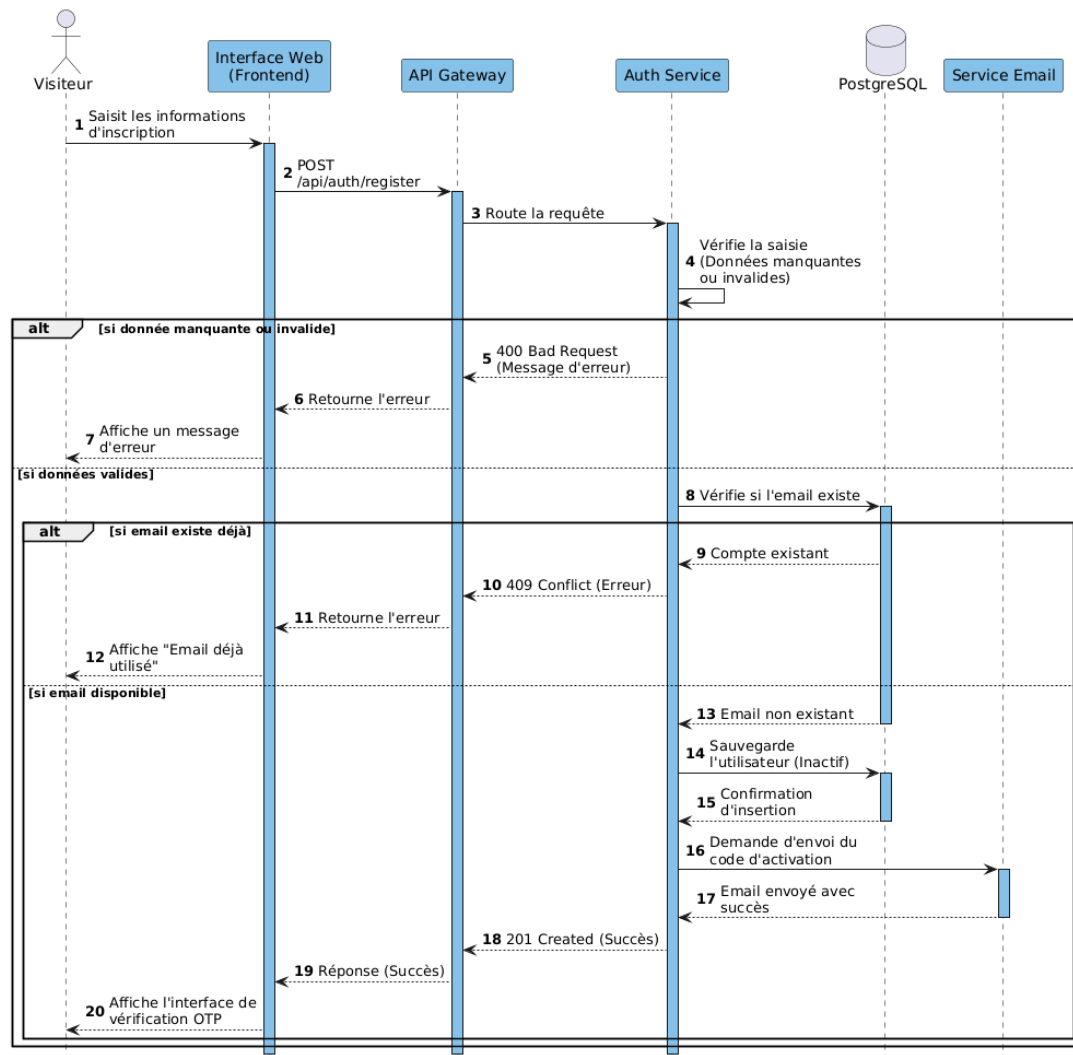


FIGURE 4.7 – Diagramme de séquence : Inscription

Diagramme de séquence « S'authentifier »

Ce diagramme illustre la validation des identifiants, la comparaison du mot de passe haché, la génération asynchrone du JWT, et le retour des informations utilisateur nécessaires à la redirection côté client.

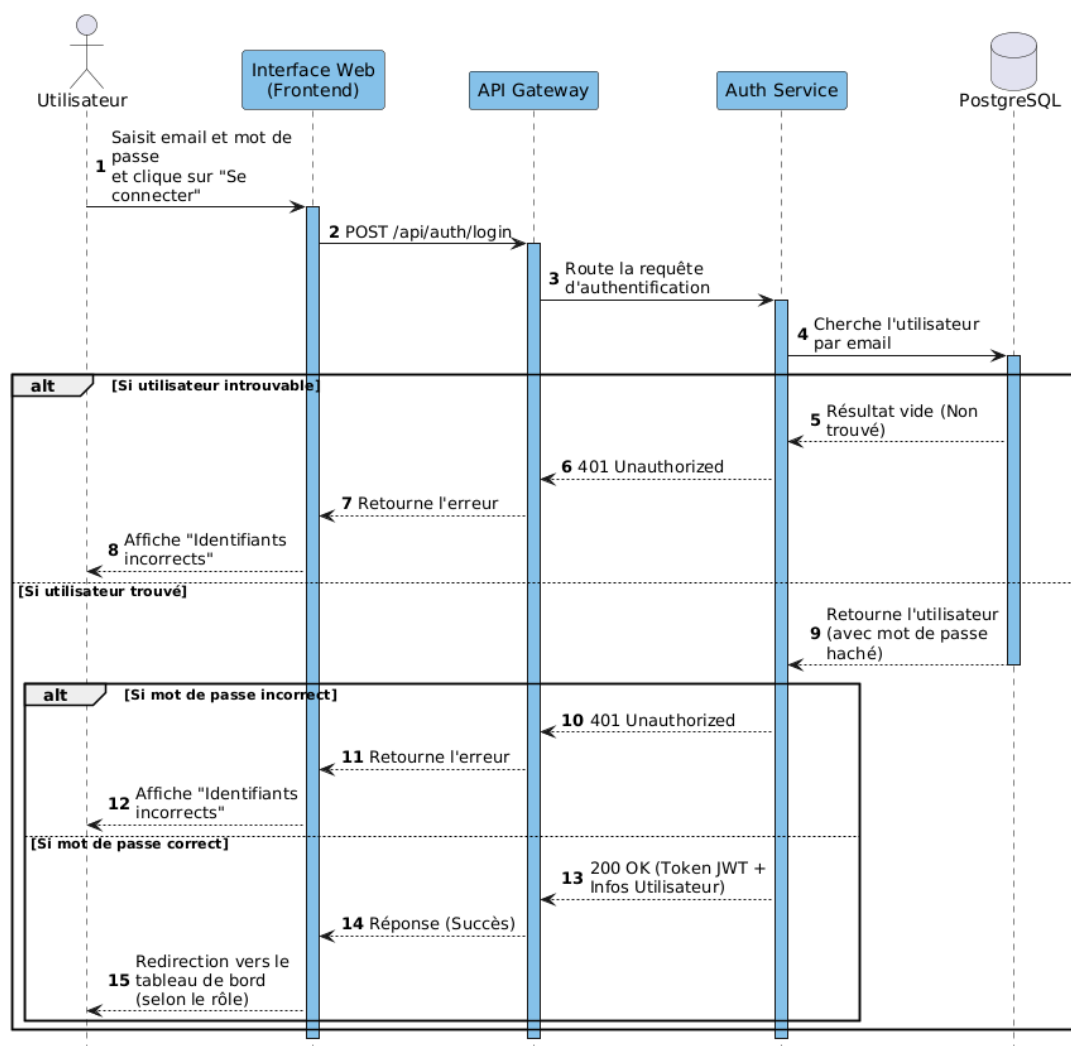


FIGURE 4.8 – Diagramme de séquence : Connexion

Diagramme de séquence « Réinitialiser son mot de passe »

Ce diagramme décrit le flux de réinitialisation du mot de passe en deux étapes. La première étape déclenche la génération et l'envoi du code OTP par email. La seconde valide ce code et applique le nouveau mot de passe haché en base de données.

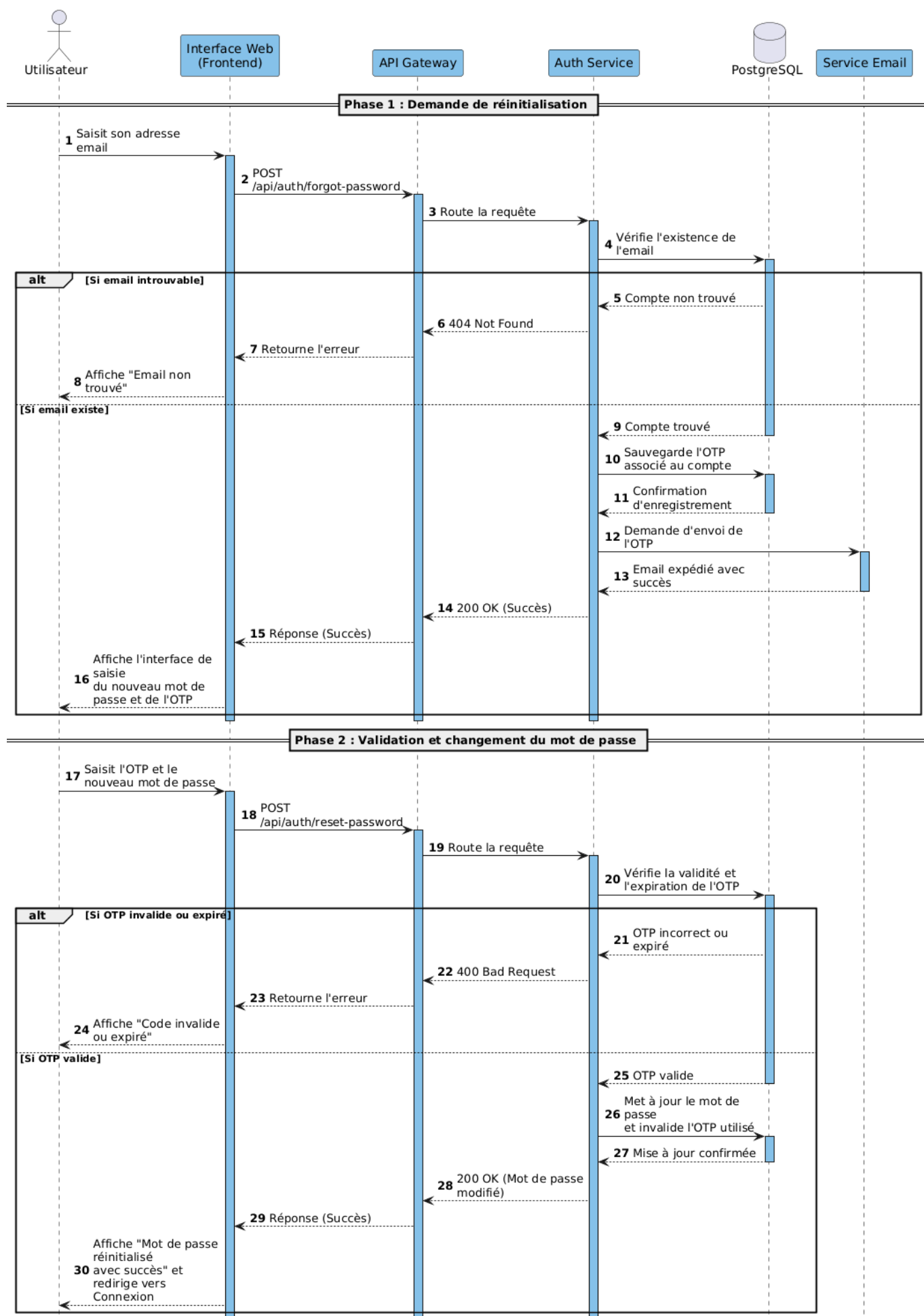


FIGURE 4.9 – Diagramme de séquence : Réinitialisation du mot de passe

Diagramme de séquence « Gérer son profil (Modification et Avatar) »

Ce diagramme modélise les opérations de mise à jour des données personnelles. Il met particulièrement en évidence le flux d'upload de l'avatar : l'image est envoyée de manière sécurisée vers le stockage cloud, qui retourne une URL publique persistée ensuite dans la base de données par le Auth Service, garantissant ainsi un profil toujours à jour.

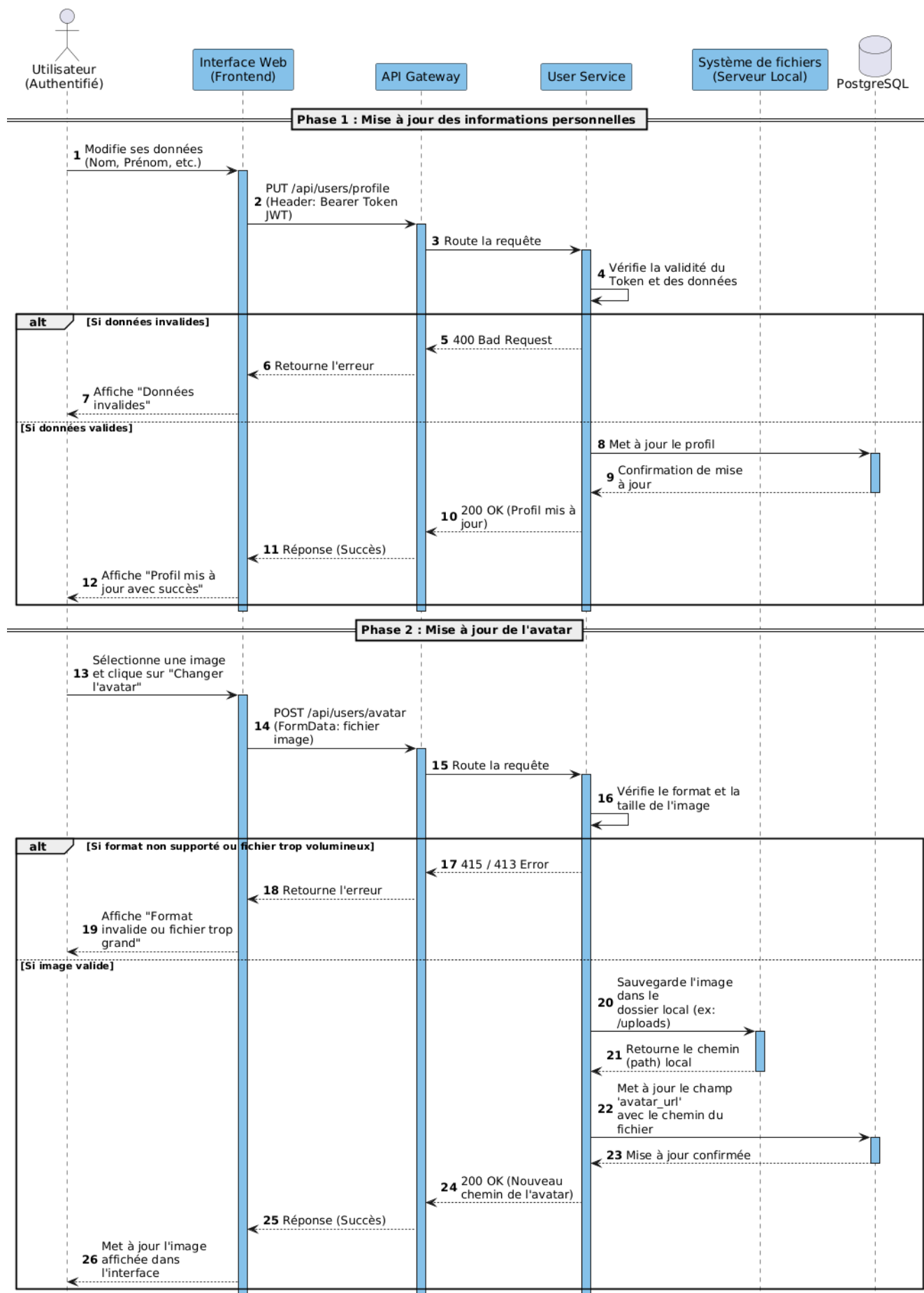


FIGURE 4.10 – Diagramme de séquence : Gestion du profil et Avatar

Diagramme de classes du Sprint 1

Le diagramme de classes de ce premier sprint se concentre sur les entités fondatrices du système : `User`, `Profile` et `OtpVerification`. Le modèle `User` encapsule la logique d'authentification et le rôle. Le modèle `Profile` porte les informations personnelles de l'utilisateur, notamment son prénom, son nom et son avatar.

FIGURE 4.11 – Diagramme de classes du Sprint 1

1.4 Réalisation et Interfaces Graphiques

Interface d'inscription et de connexion

Le premier livrable tangible de ce sprint est le couple de pages d'inscription et de connexion. L'interface d'inscription recueille les informations essentielles avec une validation en temps réel des champs. La page de connexion permet un accès rapide à la plateforme, avec un lien direct vers la page de réinitialisation du mot de passe. En cas de succès, le token JWT est stocké côté client et l'utilisateur est redirigé automatiquement vers l'espace correspondant à son rôle.

The figure displays two screenshots of the TechKids Hub web application interface, which features a dark theme and a purple accent color.

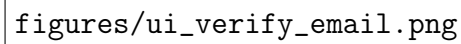
The top screenshot shows the 'Créer un compte' (Create an account) page. At the top, the TechKids logo is visible. Below it, the heading 'Créer un compte' is followed by the subtext 'Rajoutez TechKids Hub et commencez à apprendre.' There are two selection buttons: 'Parent' (Je supervise mon enfant) and 'Étudiant' (J'apprends par moi-même), with the latter being selected. Below these are input fields for 'PRÉNOM' (containing 'Mohammed') and 'NOM' (containing 'Benali'). The 'ADRESSE EMAIL' field contains 'vous@exemple.com', and the 'MOT DE PASSE' field is masked with dots. A large purple button labeled 'Créer mon compte Étudiant' is at the bottom, with a link 'Déjà un compte ? Se connecter' below it. The footer reads '© 2026 TechKids Hub - Plateforme éducative'.

The bottom screenshot shows the 'Connexion' (Login) page. It features the TechKids logo and the heading 'Connexion' with the subtext 'Bienvenue ! Entrez vos identifiants.' Below this are input fields for 'ADRESSE EMAIL' (containing 'vous@exemple.com') and 'MOT DE PASSE' (masked with dots). A link 'Mot de passe oublié ?' is next to the password field. A large purple button labeled 'Se connecter' is at the bottom, with a link 'Pas encore de compte ? Créer un compte' below it. The footer also reads '© 2026 TechKids Hub - Plateforme éducative'.

FIGURE 4.12 – Interfaces d’inscription et de connexion

Interface de vérification email

Après l’inscription, l’utilisateur est redirigé vers une page dédiée à la saisie de son code OTP. L’interface est conçue pour rester minimaliste, avec un champ de saisie centré, un bouton de validation et un lien de renvoi du code en cas d’expiration.



figures/ui_verify_email.png

FIGURE 4.13 – Interface de vérification email par OTP

Interface de réinitialisation du mot de passe

La page de réinitialisation du mot de passe se décompose en deux étapes successives, clairement indiquées par un indicateur de progression visuel.

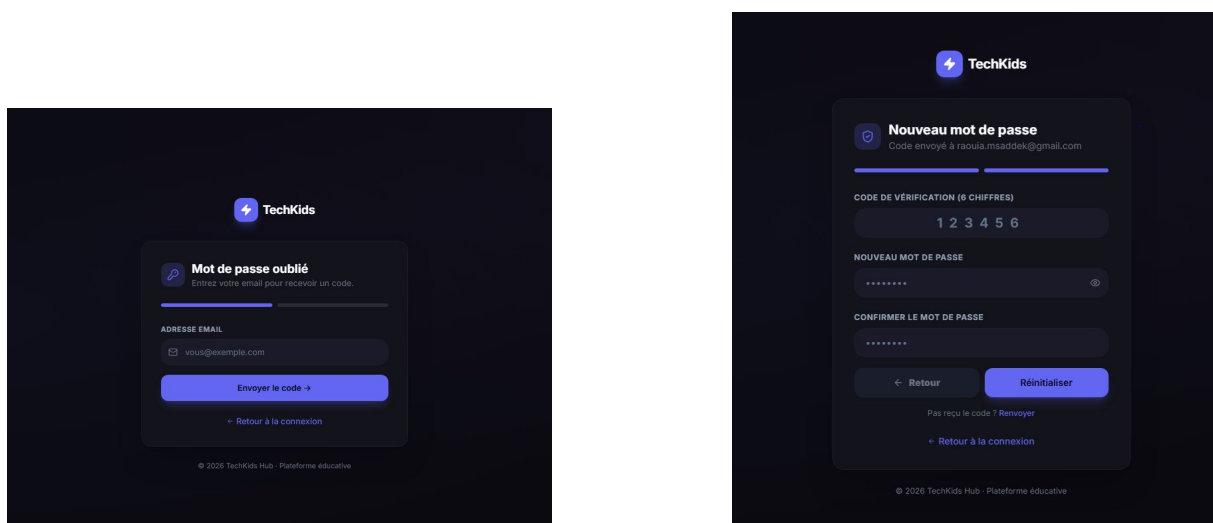


FIGURE 4.14 – Interface de réinitialisation du mot de passe (Étape 1 et 2)

Interface de gestion du profil

La page de profil constitue l'espace personnel de chaque utilisateur authentifié. Elle regroupe, au sein d'une interface épurée, toutes les fonctionnalités de gestion de l'identité numérique : consultation et mise à jour des informations, changement de mot de passe sécurisé, gestion de l'avatar et suppression du compte.

The screenshot displays the 'TechKids Hub' user profile management interface. The header includes the 'TechKids Hub' logo, a 'Catalogue de cours' link, and the user's name 'Raouia Msaddek' with a dropdown arrow. The main content area is divided into three sections:

- Informations Personnelles:** This section allows users to update their contact information. It includes input fields for 'Prénom' (Raouia), 'Nom' (Msaddek), 'Téléphone' (+216 20 000 000), and 'Adresse' (Votre adresse). A blue button labeled 'ENREGISTRER LES MODIFICATIONS' is positioned below these fields.
- Sécurité & Mot de passe:** This section is for managing account security. It includes input fields for 'Mot de passe actuel' and 'Nouveau mot de passe'. A blue button labeled 'MODIFIER LE MOT DE PASSE' is located below these fields.
- Zone dangereuse:** This section contains a warning message: 'La suppression du compte est irréversible et efface toutes vos données.' A red button labeled 'SUPPRIMER LE COMPTE' is located to the right of the warning.

On the left side of the profile management area, there is a user card for 'Raouia Msaddek' with the role 'STUDENT'. It features a profile picture, an email address 'raouia.msaddek@gmail.com', and a red button labeled 'RETIRER LA PHOTO'.

FIGURE 4.15 – Interface de gestion du profil

1.5 Conclusion du Sprint 1

Ce premier sprint a permis d'ériger le socle d'authentification de TechKids Hub. Le système prend désormais en charge l'intégralité du cycle d'identité : création de compte avec inscription différenciée par rôle, vérification email par OTP, connexion sécurisée via JWT et redirection intelligente vers l'espace approprié, réinitialisation du mot de passe oublié par code OTP en deux étapes sécurisées, et gestion complète du profil personnel. L'ensemble de ces fonctionnalités est partagé par tous les rôles de la plateforme et constitue le socle commun indispensable à la suite du développement.

Cette base fonctionnelle constitue le prérequis pour les sprints suivants. Le Sprint 2 exploitera cette infrastructure pour construire l'espace de l'apprenant, incluant le catalogue de cours, le suivi de progression et l'accès au Virtual Lab.

2 Sprint 2 : Espace Apprenant (Student Workspace)

2.1 Introduction et Sprint Backlog

Le Sprint 2 prolonge le socle posé lors du Sprint 1 en construisant les espaces fonctionnels destinés aux deux acteurs centraux du parcours d'apprentissage : l'étudiant et le parent. Côté étudiant, il s'agit de permettre la consultation du catalogue, l'inscription aux cours, le suivi de la progression et l'accès à l'interface du Virtual Lab. Côté parent, ce sprint implémente le tableau de bord analytique ainsi que la gestion des comptes enfants liés.

Ce sprint matérialise les User Stories **US_03** et **US_04** du Product Backlog global. Il s'appuie sur une architecture microservices impliquant principalement l'API Gateway, l'Auth Service (pour la liaison parent-enfant) et l'Edu-Tracker Service.

TABLE 4.8 – Sprint Backlog du Sprint 2 (Espace Apprenant et Parent)

ID	User Story (Fonctionnalité)	ID	Tâche technique	Estimation
1	Consulter le catalogue des cours .	1.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		1.2	Développer l'API et l'interface "Catalogue"	
		1.3	Tester la fonctionnalité "Catalogue"	

ID	User Story (Fonctionnalité)	ID	Tâche technique	Estimation
2	S'inscrire à un cours (création d'un Enrollment actif).	2.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		2.2	Développer la logique d'inscription	
		2.3	Tester la fonctionnalité d'inscription	
3	Suivre sa progression par module et obtenir un certificat.	3.1	Réaliser les diagrammes de cas d'utilisation et séquences	3 Jours
		3.2	Développer le calcul de progression et la certification	
		3.3	Tester la mise à jour de la progression	
4	Accéder à l'interface du Virtual Lab (shell UI et éditeur).	4.1	Réaliser les diagrammes de cas d'utilisation et séquences	5 Jours
		4.2	Développer l'interface interactive du Lab	
		4.3	Tester le bon fonctionnement de l'éditeur	
5	Lier et gérer les comptes enfants (relation parent-enfant).	5.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		5.2	Développer la logique de liaison de comptes	
		5.3	Tester la relation parent-enfant	

ID	User Story (Fonction- nalité)	ID	Tâche technique	Estimation
6	Consulter le dashboard analytique (progression, certificats).	6.1	Réaliser les diagrammes de cas d'utilisation et séquences	2 Jours
		6.2	Développer l'interface d'agrégation de données	
		6.3	Tester l'affichage du Dash- board parent	
Total Estimé				14 Jours

2.2 Analyse du Sprint et Spécifications

Diagramme cas d'utilisation global du Sprint 2

Le diagramme de cas d'utilisation du Sprint 2 représente les deux parcours principaux couverts par cette itération. Le premier suit l'étudiant depuis la consultation du catalogue jusqu'à l'accès au Virtual Lab, en passant par l'inscription et le suivi de progression. Le second modélise le flux parent, depuis la liaison d'un compte enfant jusqu'à la consultation du tableau de bord.

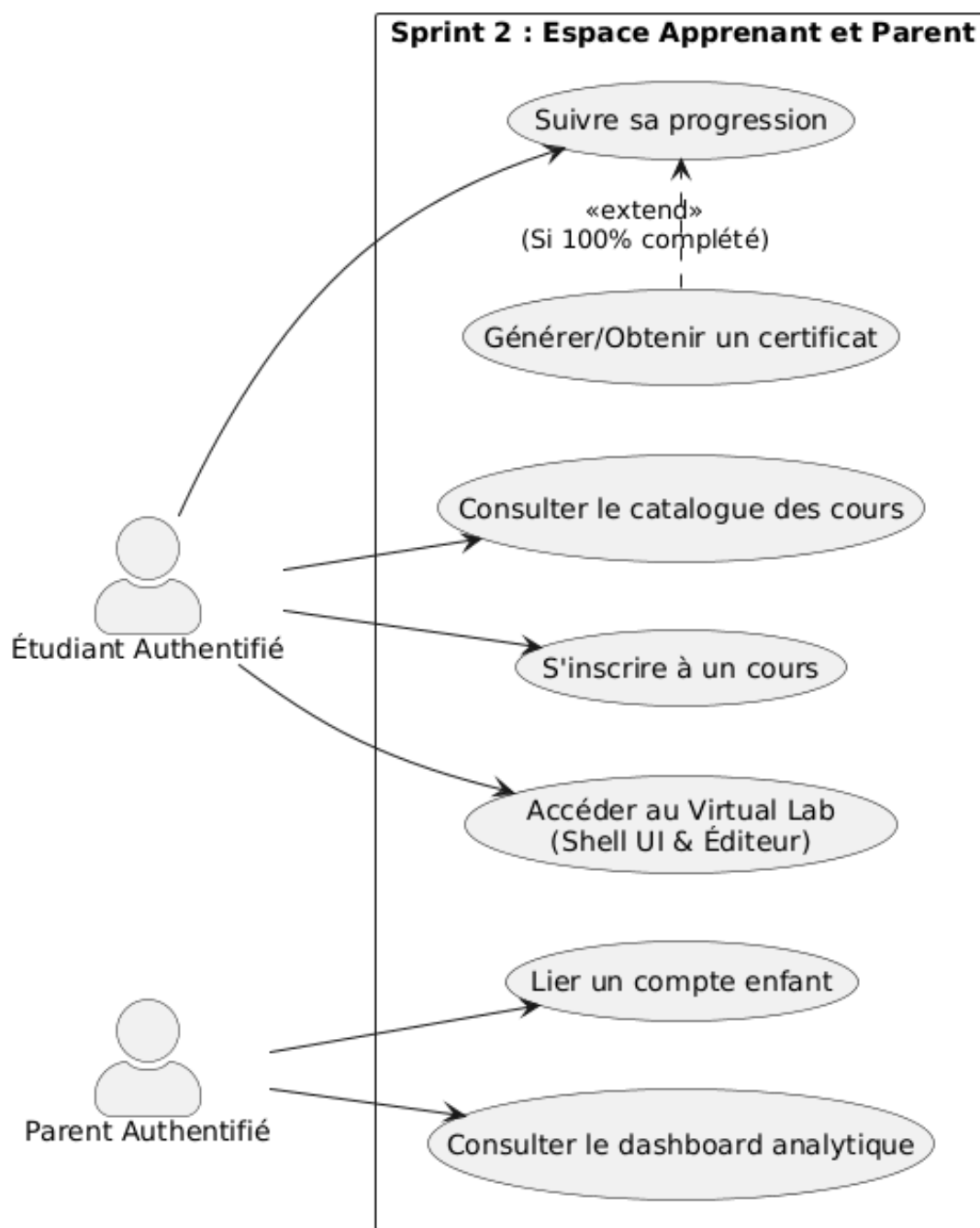


FIGURE 4.16 – Diagramme cas d'utilisation global du Sprint 2

Raffinement du cas d'utilisation « Consulter le catalogue des cours »

Le cas d'utilisation « Consulter le catalogue » permet à l'étudiant authentifié de parcourir l'ensemble des cours disponibles sur la plateforme.

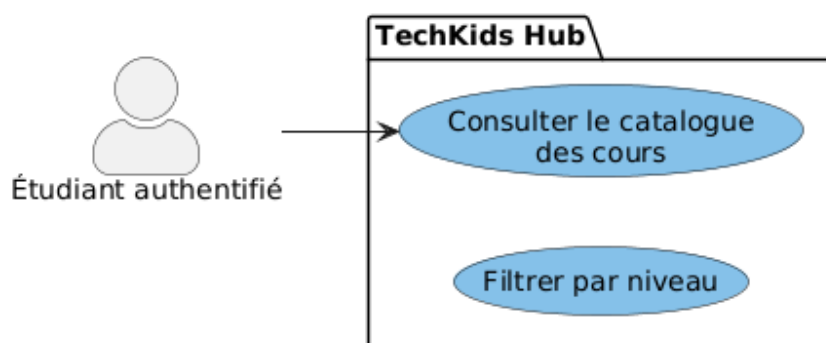


FIGURE 4.17 – Diagramme de cas d'utilisation « Consulter le catalogue »

TABLE 4.9 – Description textuelle : Consulter le catalogue des cours

Champ	Description
Nom	Consulter le catalogue des cours
Acteur principal	Étudiant authentifié
Préconditions	L'utilisateur est connecté avec le rôle STUDENT.
Postconditions	La liste des cours publiés est affichée avec leur niveau et statut d'inscription.
Scénario nominal	1. L'étudiant accède à <code>/courses</code>. 2. Le front appelle <code>GET /api/courses</code>. 3. L'Edu-Tracker Service retourne les cours publiés. 4. Les cours auxquels l'étudiant est déjà inscrit affichent un badge de progression.
Scénarios alternatifs	Alt 1 : Aucun cours disponible → Message "Aucun cours publié".
Règles de gestion	RG1 : Seuls les cours avec <code>isPublished = true</code> sont visibles.

Raffinement du cas d'utilisation « S'inscrire à un cours »

Le cas d'utilisation « S'inscrire à un cours » permet à l'étudiant de rejoindre un parcours pédagogique. L'inscription crée un enregistrement `Enrollment` en base de données et initialise la progression de l'étudiant pour chaque module du cours.

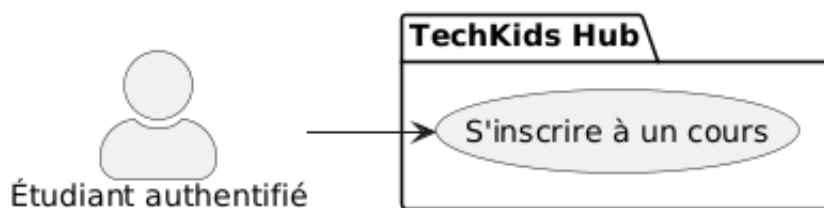


FIGURE 4.18 – Diagramme de cas d'utilisation « s'inscrire au cours »

TABLE 4.10 – Description textuelle : S'inscrire à un cours

Champ	Description
Nom	S'inscrire à un cours
Acteur principal	Étudiant authentifié
Préconditions	L'étudiant n'est pas déjà inscrit à ce cours.
Postconditions	Un Enrollment est créé avec statut ACTIVE . Une Progression est initialisée pour chaque module.
Scénario nominal	1. L'étudiant clique sur « S'inscrire » sur la fiche cours. 2. Le front appelle <code>POST /api/enrollments</code>. 3. L'Edu-Tracker vérifie l'unicité de la paire <code>[studentId, courseId]</code>. 4. Un Enrollment est créé. 5. Une Progression <code>NOT_STARTED</code> est créée pour chaque module. 6. L'étudiant est redirigé vers la page du cours.
Scénarios alternatifs	Alt 1 : Déjà inscrit → Bouton désactivé + message.
Règles de gestion	RG1 : Un étudiant ne peut s'inscrire qu'une seule fois par cours (contrainte unique).

Raffinement du cas d'utilisation « Suivre sa progression »

Ce cas d'utilisation permet à l'étudiant de visualiser son avancement dans un cours, module par module. L'avancement est mis à jour à chaque interaction avec le contenu. (Note : ce cas d'utilisation inclut une extension vers l'obtention automatique d'un certificat).

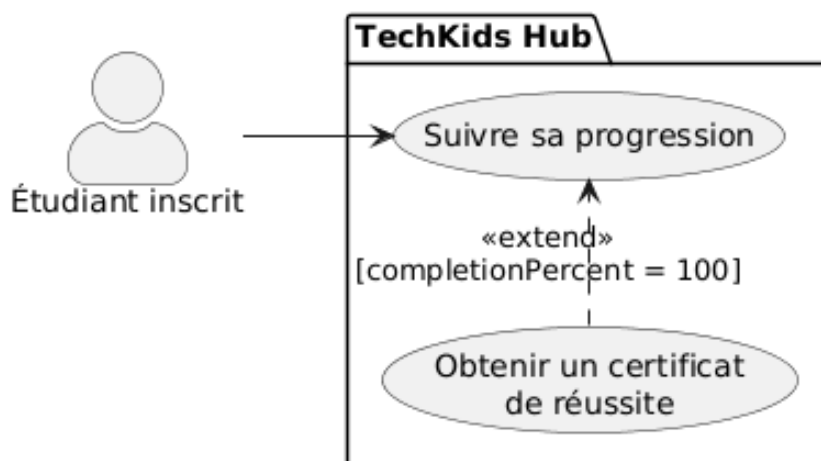


FIGURE 4.19 – Diagramme de cas d'utilisation « Consulter le catalogue »

TABLE 4.11 – Description textuelle : Suivre sa progression

Champ	Description
Nom	Suivre sa progression
Acteur principal	Étudiant inscrit à un cours
Préconditions	L'étudiant est inscrit au cours. Au moins une <i>Progression</i> existe en base.
Postconditions	L'affichage reflète l'état réel de la progression en base.
Scénario nominal	1. L'étudiant accède à son dashboard ou à la page cours. 2. Le front appelle. 3. L'Edu-Tracker retourne la liste des <i>Progressions</i> par module. 4. Un indicateur visuel (barre, %) est affiché pour chaque module. 5. Un récapitulatif global du cours est calculé.
Règles de gestion	RG1 : <i>completionPercent</i> est calculé côté backend. RG2 : Statut <i>COMPLETED</i> déclenché quand <i>completionPercent</i> = 100. RG3 : Lorsque le statut passe à <i>COMPLETED</i>, un certificat de réussite est généré automatiquement.

Raffinement du cas d'utilisation « Accéder au Virtual Lab »

Le Virtual Lab est l'espace de pratique centrale de TechKids Hub. Il propose à l'étudiant un éditeur de code interactif pour écrire et exécuter ses scripts, ainsi qu'un environnement

de simulation de circuits électroniques.

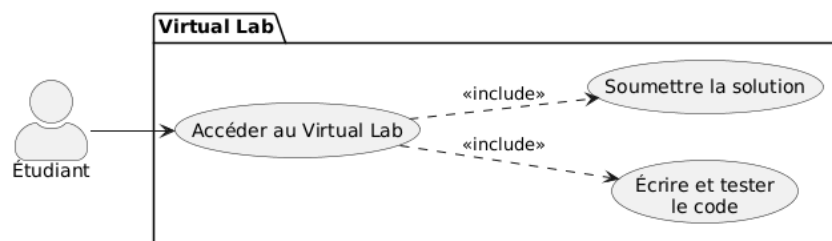


FIGURE 4.20 – Diagramme de cas d'utilisation « Consulter le catalogue »

TABLE 4.12 – Description textuelle : Accéder au Virtual Lab

Champ	Description
Nom	Accéder au Virtual Lab
Acteur principal	Étudiant inscrit
Préconditions	L'étudiant accède à un exercice de type <code>CODE_CHALLENGE</code> ou <code>CIRCUIT_BUILD</code> .
Postconditions	L'interface du Virtual Lab est chargée avec l'exercice associé.
Scénario nominal	1. L'étudiant clique sur un exercice dans son module. 2. Le front charge l'interface du Virtual Lab. 3. L'énoncé de l'exercice est affiché. 4. L'étudiant écrit son code dans l'éditeur. 5. Il peut soumettre sa solution pour évaluer son travail.
Scénarios alternatifs	Alt 1 : Exercice de type <code>QUIZ</code> → Interface QCM à la place du Lab.
Règles de gestion	RG1 : L'interface s'adapte dynamiquement au type d'exercice.

Raffinement des cas d'utilisation liés aux Parents

L'espace Parent permet de lier des comptes enfants et de visualiser un tableau de bord analytique de leurs performances.

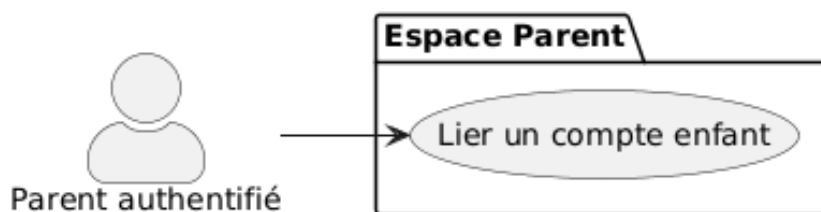


FIGURE 4.21 – Diagramme de cas d'utilisation « Consulter le catalogue »

TABLE 4.13 – Description textuelle : Lier et gérer les comptes enfants

Champ	Description
Nom	Lier un compte enfant
Acteur principal	Parent authentifié
Préconditions	Le compte enfant existe en base et possède le rôle STUDENT .
Scénario nominal	1. Le parent accède à "Mes enfants" et saisit l'email de l'enfant. 2. Le front appelle <code>POST /api/parent/link-child</code>. 3. Le champ <code>parentId</code> est renseigné sur le compte enfant en base. 4. L'enfant apparaît dans la liste du parent.
Règles de gestion	RG1 : Un STUDENT ne peut avoir qu'un seul parent lié.

2.3 Conception Technique et Diagrammes

Afin de ne présenter que les flux les plus pertinents, nous modélisons ci-dessous les diagrammes de séquence des processus impliquant des interactions multi-services ou une logique métier complexe.

Diagramme de séquence : S'inscrire à un cours

Ce diagramme détaille la création de l'Enrollment et l'initialisation atomique des Progressions pour chaque module du cours. Il illustre la transaction garantissant la cohérence entre ces deux opérations.

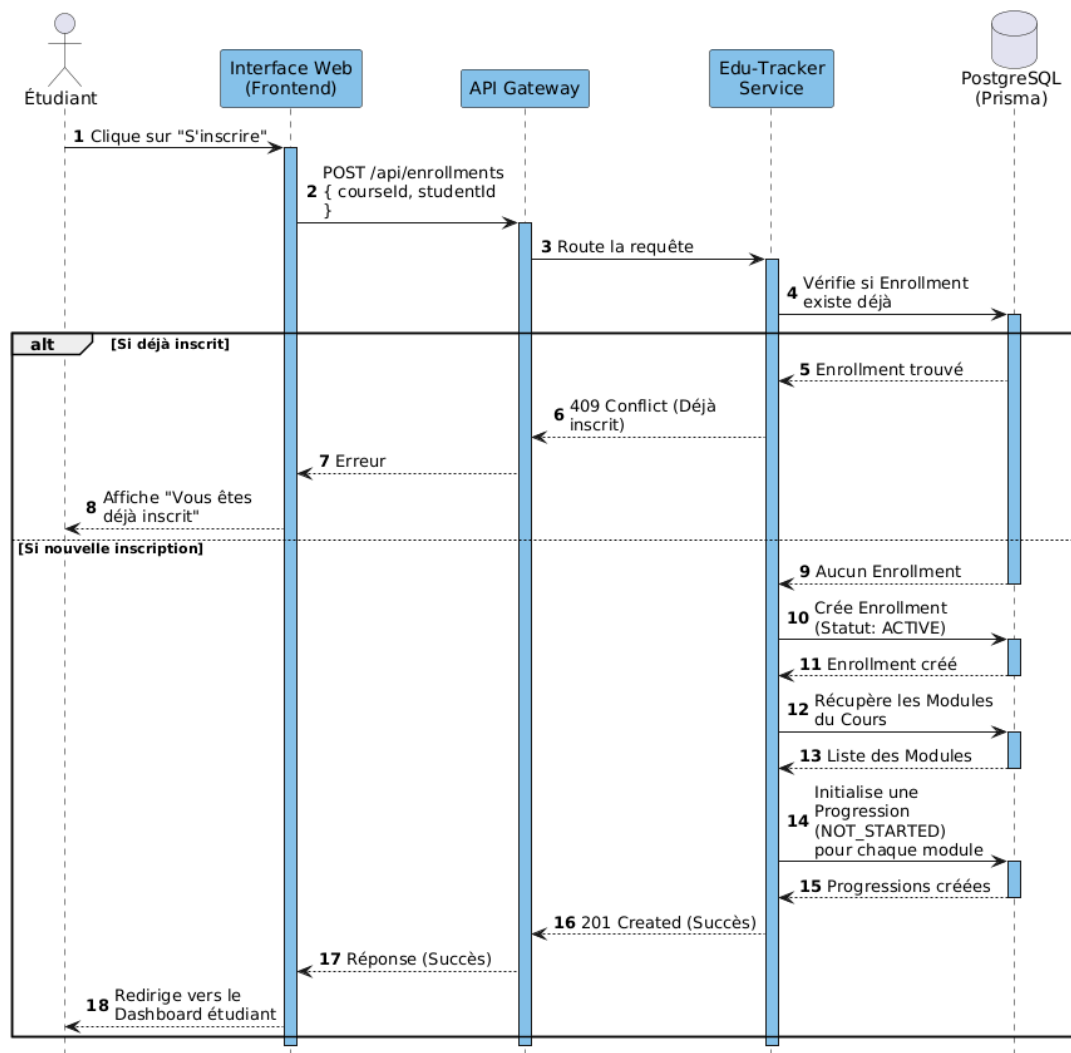


FIGURE 4.22 – Diagramme de séquence : Inscription à un cours

Diagramme de séquence : Suivre sa progression

Ce diagramme modélise la récupération et l'agrégation des Progressions par module, puis le calcul du pourcentage global de complétion retourné au frontend.

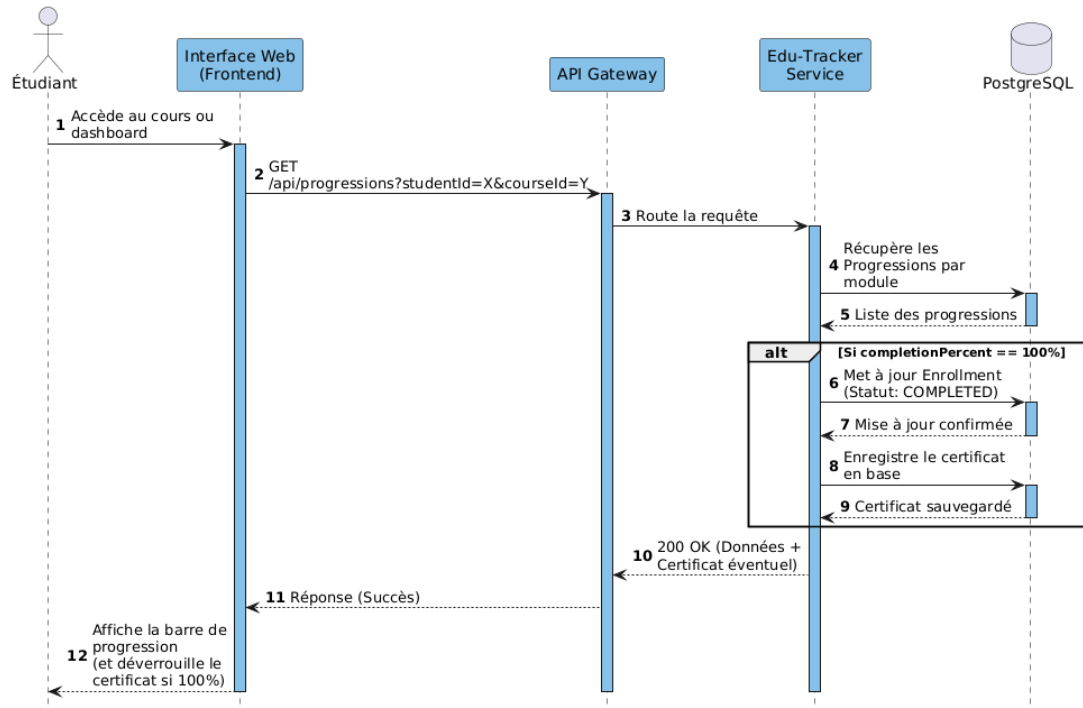


FIGURE 4.23 – Diagramme de séquence : Suivi de progression

Diagramme de séquence : Consulter le dashboard parent

Ce diagramme illustre la jointure multi-tables réalisée côté backend pour agréger les données de progression, d'enrollment et de soumissions d'un enfant, avant de les retourner formatées au dashboard parent.

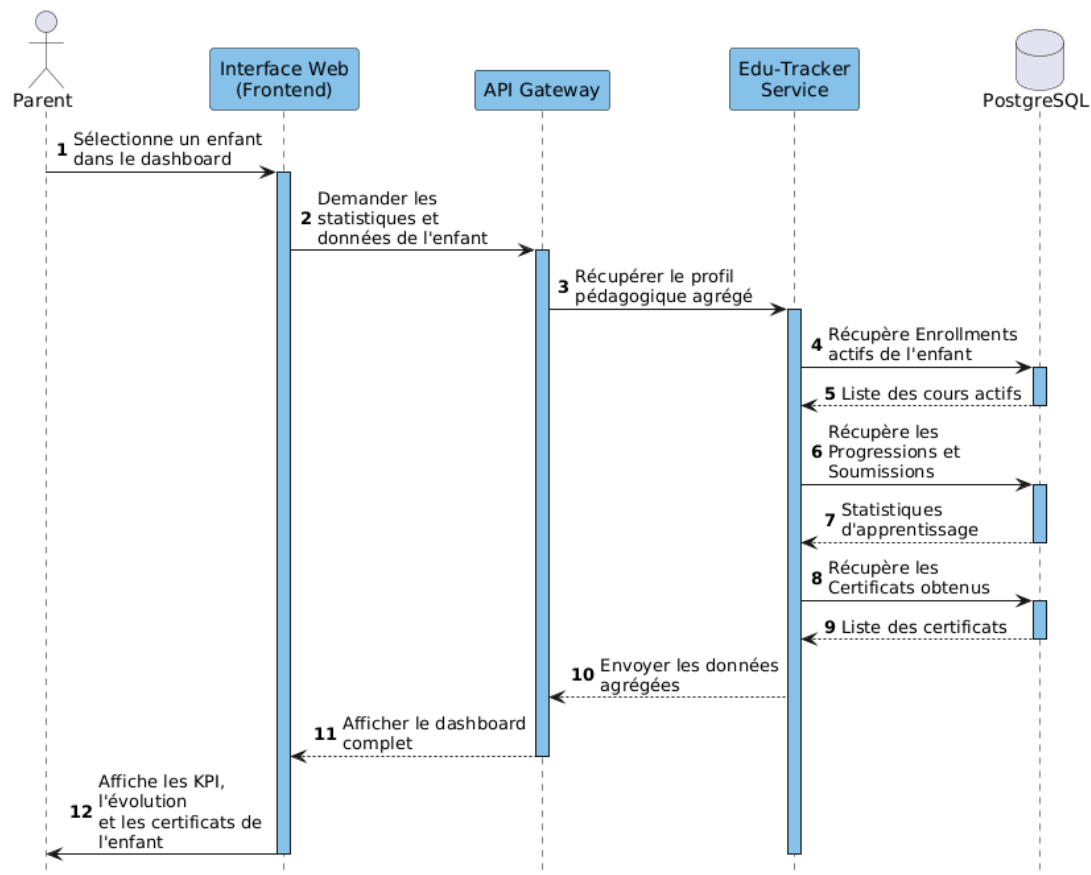


FIGURE 4.24 – Diagramme de séquence : Dashboard parent

Diagramme de classes du Sprint 2

Le diagramme de classes de ce sprint complète le modèle du Sprint 1 en introduisant les entités du domaine pédagogique : *Course*, *Module*, *Exercise*, *Enrollment*, *Progression* et *Submission*.

FIGURE 4.25 – Diagramme de classes : Sprint 2

2.4 Réalisation et Interfaces Graphiques

Interface du catalogue des cours

La page catalogue présente l'ensemble des cours publiés sous forme de cartes visuelles. Chaque carte affiche le titre, le niveau, l'enseignant et le statut d'inscription de l'étudiant connecté. Un système de filtres par niveau permet d'affiner la recherche.

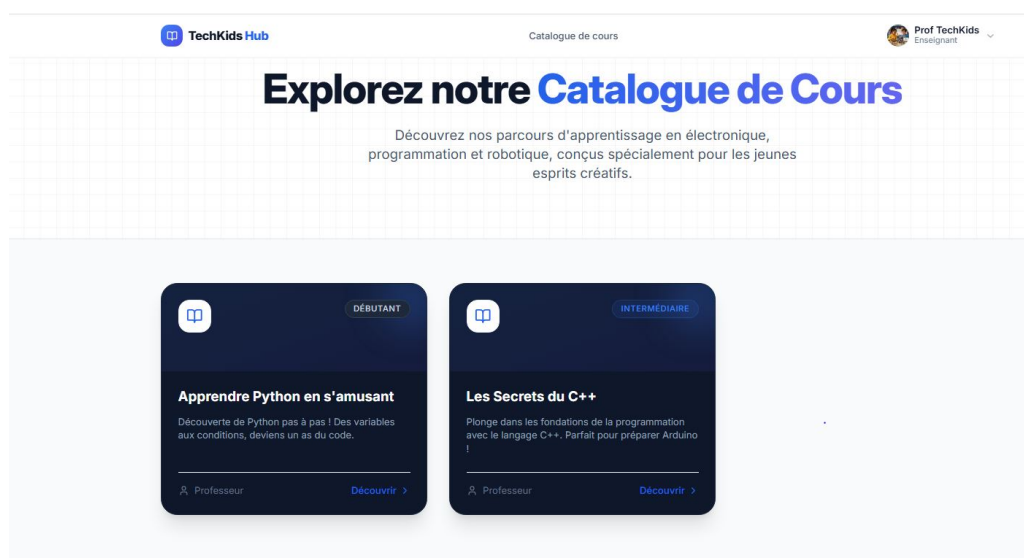


FIGURE 4.26 – Interface du catalogue des cours

Interface du dashboard étudiant

Le tableau de bord de l'étudiant centralise ses cours actifs, sa progression globale par parcours et ses dernières activités. Chaque cours inscrit affiche une barre de progression calculée à partir des Progressions de ses modules.

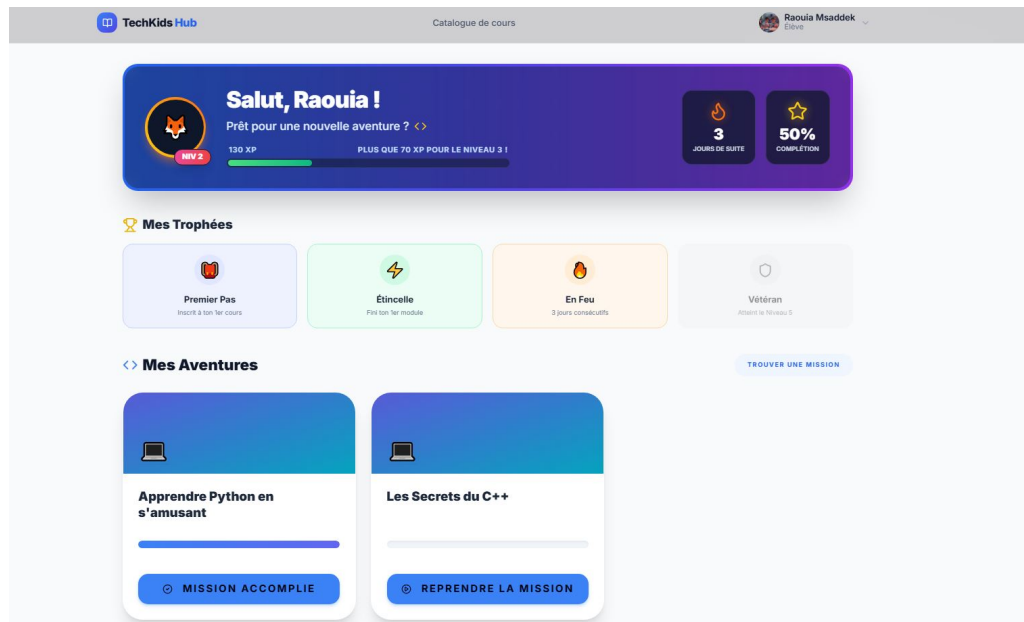


FIGURE 4.27 – Interface du dashboard étudiant

Interface du certificat de réussite

À l'issue de la complétion d'un cours (100%), un certificat de réussite numérique est automatiquement généré et déverrouillé dans l'espace étudiant. Il atteste des compétences

acquises et valorise l'assiduité de l'apprenant.

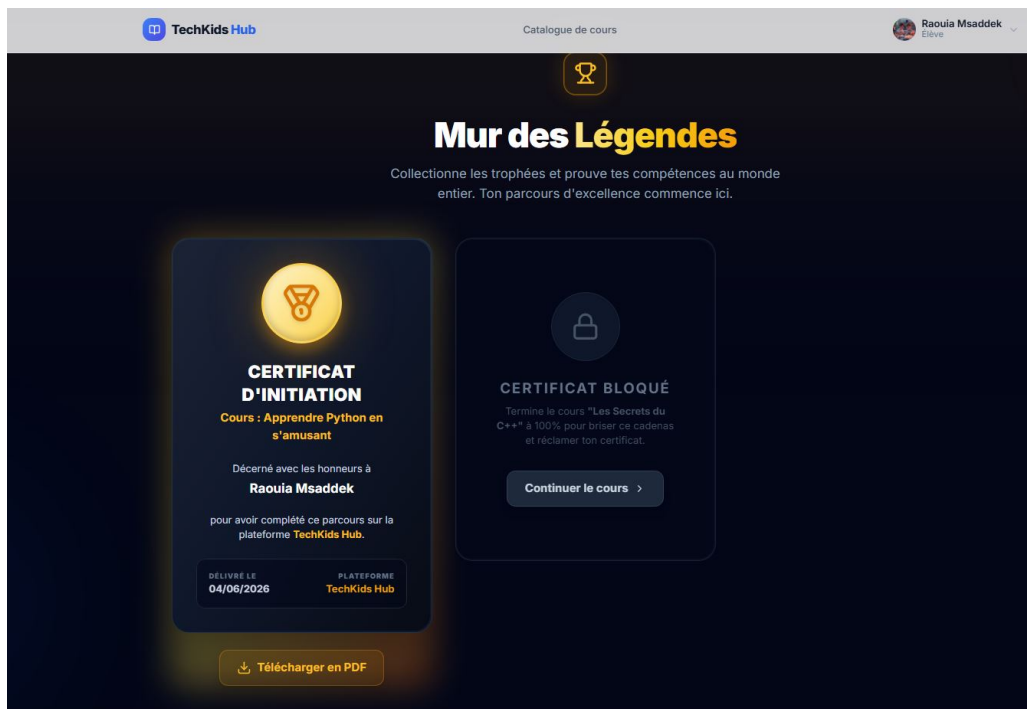


FIGURE 4.28 – Interface du certificat de réussite

Interface du Virtual Lab

L'interface du Virtual Lab est divisée en deux panneaux principaux. Le panneau gauche affiche l'énoncé de l'exercice et les instructions. Le panneau droit intègre l'éditeur de code avec coloration syntaxique. Un bouton de soumission permet de valider le travail de l'étudiant.

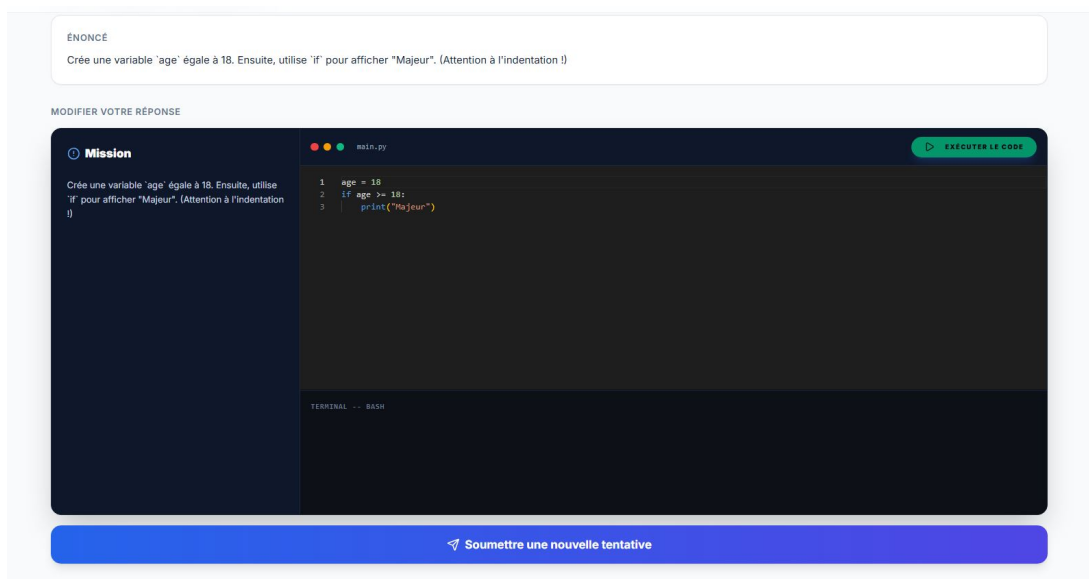


FIGURE 4.29 – Interface du Virtual Lab (Shell UI)

Interface du dashboard parent

Le dashboard parent offre une vue analytique claire de la progression des enfants liés. Il affiche les cours actifs, les pourcentages de complétion, l'historique des dernières activités, ainsi qu'une section dédiée aux certificats de réussite obtenus par l'enfant, valorisant ainsi ses accomplissements.

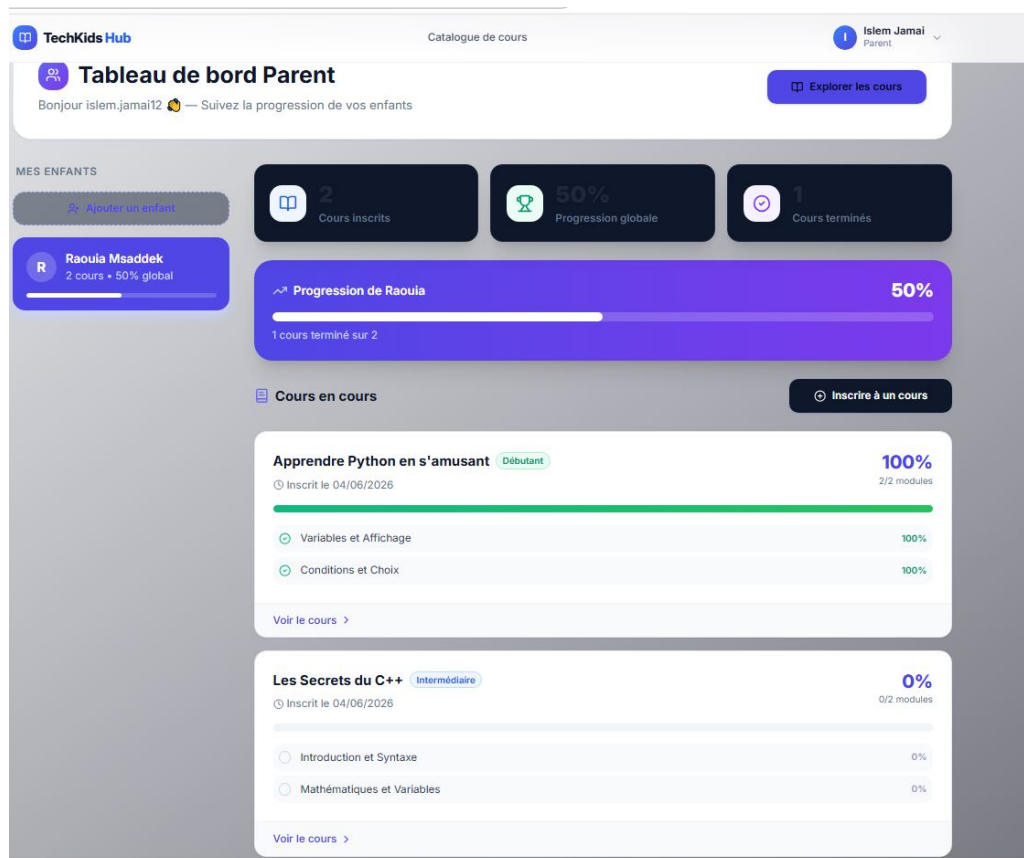


FIGURE 4.30 – Interface du dashboard parent

2.5 Conclusion du Sprint 2

Le Sprint 2 a permis de concrétiser les deux espaces fonctionnels au cœur de la valeur pédagogique de TechKids Hub. L'étudiant dispose désormais d'un catalogue navigable, d'un système d'inscription sécurisé, d'un suivi de progression granulaire et d'une interface de Virtual Lab opérationnelle. Du côté parent, l'implémentation du Dashboard analytique offre une vision claire sur l'évolution de l'enfant et valorise ses accomplissements grâce aux certificats de réussite. Ces modules préparent ainsi le terrain pour l'intégration de la gestion des contenus par les enseignants prévue dans l'itération suivante.

Bibliographie et Webographie

- [1] L'AGILISTE. *Introduction aux méthodes agiles*. Consulté le 01/06/2026. 2026. URL : <https://agiliste.fr/introduction-methodes-agiles/>.
- [2] LUCIDCHART. *Qu'est-ce que le langage UML ?* Consulté le 01/06/2026. 2026. URL : <https://www.lucidchart.com/pages/fr/langage-uml>.
- [3] AMAZON WEB SERVICES. *AWS Well-Architected Framework – Generative AI Lens*. Consulté le 01/06/2026. 2024. URL : <https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-ai-lifecycle.html>.

Résumé

Rédigez ici un court résumé de votre travail en français (environ 150 à 200 mots). Il doit résumer la problématique, la méthodologie adoptée (Agile/Scrum), et les principaux résultats obtenus.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor.

Mots-clés : PFE, Agile, Scrum, [Vos mots-clés].

Abstract

This is the English version of your summary. Ensure it is accurately translated and uses technical terminology appropriate for the field of computer science.

This Final Year Project was carried out as part of the License degree in Computer Science. It focuses on [your topic here] using an Agile approach and Scrum methodology...

Keywords : Final Year Project, Agile, Scrum, [Your keywords].

